Architetture dei Sistemi di Elaborazione

Antonio Villella

Anno accademico 2026–2027

Indice

Introduzione al Corso e Modalità d'Esame **5**
Di cosa tratta la materia . . . 5
Come è strutturato l'esame . . . 5

1 Introduction to Computer Design **7**
1.1 Sistemi Embedded e Classi di Parallelismo . . . 7
1.1.1 Classi di Parallelismo . . . 7
1.2 Architettura dei Calcolatori e Requisiti . . . 8
1.2.1 La Legge di Moore . . . 8
1.3 Affidabilità (Dependability) e Sicurezza . . . 9
1.3.1 Metriche di Affidabilità . . . 9
1.3.2 La Curva a Vasca (Bathtub Curve) e il Burn-in . . . 9
1.4 Valutazione delle Prestazioni (Performance Evaluation) . . . 10
1.4.1 I Benchmark . . . 10
1.4.2 Metriche di Confronto: Media Aritmetica e Pesata . . . 11
1.5 Miglioramenti Architetturali . . . 12
1.5.1 La Legge di Amdahl . . . 12
1.5.2 Equazione delle prestazioni della CPU . . . 12

2 Instruction Set Principles **14**
2.1 Introduzione all'Instruction Set Architecture (ISA) . . . 14
2.2 Modelli di Memorizzazione Interna . . . 14
2.3 Organizzazione della Memoria e Accesso ai Dati . . . 15
2.4 Modi di Indirizzamento (Addressing Modes) . . . 15
2.5 Operazioni nell'Instruction Set e Frequenze d'Uso . . . 16
2.6 Istruzioni di Controllo del Flusso (Control Flow) . . . 16
2.6.1 L'indirizzo di Destinazione (Destination Address) . . . 16
2.6.2 Salti a Registro (Register Indirect Jumps) e Procedure . . . 16
2.6.3 Valutazione delle Condizioni . . . 17
2.6.4 Gestione delle Procedure (Procedure Calls) . . . 17
2.7 Tipi e Dimensioni degli Operandi . . . 17
2.8 Codifica dell'Instruction Set (Instruction Set Encoding) . . . 18
2.8.1 Codifica Variabile (Es. Intel 80x86, VAX) . . . 18
2.8.2 Codifica Fissa (Es. RISC-V, ARM, MIPS) . . . 18
2.8.3 Codifica Ibrida (Es. RISC-V Compressed, ARM Thumb-2) . . . 19

3 Introduzione a RISC-V **20**
3.1 Storia e Caratteristiche Generali . . . 20
3.1.1 L'Architettura Load-Store . . . 20
3.1.2 Tipi di Dati e Formato delle Istruzioni . . . 21

3.2 Rappresentazione dei Dati . . . 21
3.2.1 Interi e Complemento a Due . . . 21
3.2.2 Floating-Point (IEEE 754) . . . 22
3.3 Organizzazione dell'Instruction Set e Registri . . . 22
3.3.1 I Registri . . . 23
3.4 Modi di Indirizzamento . . . 24
3.5 Formato delle Istruzioni . . . 24
3.6 Analisi delle Istruzioni Base . . . 25
3.6.1 Load e Store . . . 25
3.6.2 Operazioni Aritmetico-Logiche (ALU) . . . 26
3.6.3 Salti e Diramazioni (Branches e Jumps) . . . 26

4 Introduzione a gem5 e RISC-V **27**
4.1 Il Simulatore gem5 . . . 27
4.1.1 La Pipeline a 5 Stadi . . . 27
4.2 Struttura dei Programmi in Assembly . . . 28
4.3 Confronto Prestazionale: RISC vs CISC . . . 28
4.4 Ambiente di Sviluppo e Simulazione . . . 29
4.5 Stesura di un Programma: Pratiche Fondamentali . . . 30
4.5.1 Il problema del Cache Miss iniziale . . . 30
4.5.2 L'etichetta `_start` e la struttura del codice . . . 30

5 Pipelining **32**
5.1 Introduzione e Concetti Base . . . 32
5.1.1 Definizioni Fondamentali . . . 32
5.1.2 Pipeline Ideale . . . 33
5.2 Il Datapath Unpipelined (5 Stadi) . . . 33
5.3 Transizione all'Architettura Pipelined . . . 34
5.4 Pipeline Hazards (Conflitti) . . . 35
5.4.1 Data Hazards (Conflitti sui Dati) . . . 36
5.4.2 Control Hazards (Conflitti di Controllo) . . . 37

6 Operazioni Multiciclo (Multicycle Operations) **39**
6.1 Introduzione alle Operazioni Floating-Point . . . 39
6.2 Estensione della Pipeline per il Floating-Point . . . 39
6.3 Latenza e Intervallo di Inizializzazione . . . 40
6.3.1 Unità Pipelined e Non Pipelined . . . 41
6.4 Conflitti (Hazards) nelle Pipeline Multiciclo . . . 41
6.4.1 Conflitti Strutturali (Structural Hazards) . . . 42
6.4.2 Conflitti Dati e Forwarding . . . 42
6.5 Moduli e Fasi Operative del Floating-Point . . . 43
6.6 Prestazioni (Performance) . . . 43

7 Exceptions and Interrupts in Pipelined Processors **44**
7.1 Distinzione tra Eccezione e Interruzione . . . 44
7.2 Classificazione delle Eccezioni . . . 44
7.2.1 Sincrone vs. Asincrone . . . 45
7.2.2 User Requested vs. Coerced (Richieste vs. Forzate) . . . 45
7.2.3 Maskable vs. Non-maskable (Mascherabili vs. Non mascherabili) . . . 45
7.2.4 Within vs. Between Instructions (All'interno vs. Tra le istruzioni) . . . 45
7.2.5 Resume vs. Terminate (Ripresa vs. Terminazione) . . . 45

7.3 Meccanismo di Gestione nella Pipeline . . . 45
7.3.1 Protocolli di Interruzione a confronto . . . 46
7.4 Eccezioni Precise e Imprecise . . . 46
7.5 Complicazioni nella Pipeline e Ordine delle Eccezioni . . . 47
7.5.1 Eccezioni Contemporanee . . . 47
7.5.2 Eccezioni in ordine inverso . . . 48
7.5.3 Altre Complicazioni del Set di Istruzioni . . . 48

8 Instruction Level Parallelism (ILP) 50
8.1 Introduzione . . . 50
8.2 Approcci allo sfruttamento dell'ILP . . . 50
8.3 Parallelismo nei Basic Block . . . 50
8.4 Tecniche di Ottimizzazione Statica (Software) . . . 51
8.4.1 Rescheduling (Rischedulazione) . . . 51
8.4.2 Loop-Level Parallelism e Loop Unrolling . . . 53
8.5 Dipendenze e Hazard . . . 54
8.5.1 Dipendenze di Dato (Data Dependencies - RAW) . . . 54
8.5.2 Dipendenze di Nome (Name Dependencies) . . . 54
8.5.3 Register Renaming (Rinominazione dei registri) . . . 55
8.5.4 Dipendenze di Controllo . . . 55

9 Memorie Cache 56
9.1 Introduzione e Motivazione . . . 56
9.1.1 Architettura di Von Neumann vs Harvard . . . 56
9.2 Gerarchia delle Memorie e Principio di Località . . . 57
9.2.1 Principio di Località . . . 58
9.2.2 Prestazioni e Hit Ratio . . . 59
9.3 Organizzazione Interna della Cache . . . 59
9.3.1 Ricerca di un blocco dati nella Cache . . . 60
9.3.2 Posizionamento della Cache nel Sistema . . . 61
9.4 Tecniche di Mappatura (Mapping) . . . 62
9.4.1 Mappatura Diretta (Direct Mapping) . . . 62
9.4.2 Mappatura Associativa agli Insiemi (Set Associative) . . . 62
9.4.3 Mappatura Completamente Associativa (Fully Associative) . . . 63
9.5 Algoritmi di Rimpiazzamento . . . 63
9.5.1 Approccio FIFO con Seconda Chance (Clock Bit) . . . 63
9.5.2 Algoritmo pseudo-LRU (pLRU) . . . 64
9.6 Gestione delle Scritture e Coerenza . . . 64
9.6.1 Aggiornamento della Memoria Principale . . . 64
9.6.2 Coerenza della Memoria (Multiprocessore) e Protocollo MESI . . . 65
9.7 Esempi Calcolati e Strutturazione Fisica . . . 65
9.7.1 Esempio 1: Cache Direct Mapped . . . 65
9.7.2 Esempio 2: Cache Set-Associative in un Layout di Memoria Misto . . . 66
9.7.3 Esempio 3: Cache Associativa a 4-vie . . . 67

10 Branch prediction 69

11 Dynamic scheduling 70

12 Hardware speculation 71

13 Sistemi di ingresso e uscita **72**
13.1 Periferiche 72
13.2 Interrupt 72
13.3 Accesso diretto alla memoria 72

14 Esercizi **73**
14.1 Esercizi svolti 73
14.2 Esercizi da svolgere 73

15 VLIW **74**

16 Riepilogo **75**

17 ESERCIZI **76**

18 Esercitazione: Analisi delle Prestazioni e Pipeline **77**
18.1 Definizione dell'Architettura del Processore 77
18.2 Analisi del Codice Assembly 78
18.3 Calcolo dei Colpi di Clock 78
18.3.1 Fase di Inizializzazione (Fuori dal ciclo) 79
18.3.2 Tracciamento di una Iterazione del Ciclo (`loop`) 79
18.3.3 Calcolo del Tempo Totale di Esecuzione 81
18.4 Analisi degli Hazard (Domande Teoriche) 81
18.4.1 Hazard sul registro `x7` 81
18.4.2 Hazard sul registro `F6` 82

Introduzione al Corso e Modalità d'Esame

Di cosa tratta la materia

Il corso di Architetture dei Sistemi di Elaborazione è un insegnamento obbligatorio del primo anno per la Laurea Magistrale in Ingegneria Informatica. L'obiettivo principale della materia è fornire agli studenti una conoscenza di base sull'architettura dei moderni sistemi basati su microprocessore, con particolare attenzione al core del processore. Il corso analizza le diverse componenti di un sistema di elaborazione, partendo dall'architettura interna dei microprocessori fino ad arrivare alla gestione delle periferiche.

Il programma è strutturato in due macro-blocchi principali:

- **Architettura dei processori moderni:** questa parte include un'introduzione al design dei computer e analizza l'architettura, il comportamento e le prestazioni dei processori RISC, CISC e superscalari. Vengono inoltre approfonditi temi fondamentali come le memorie cache, la memoria virtuale, le pipeline, gli hazard e le tecniche di ottimizzazione statica e dinamica.
- **Sistemi basati su ARM:** questa sezione si concentra sull'architettura di base dei processori ARM, sul linguaggio Assembly e sui sistemi di emulazione. Gli studenti imparano il flusso di sviluppo per sistemi embedded utilizzando schede di sviluppo come la board Landtiger basata su System-on-Chip ARM Cortex-M3 (tramite l'ambiente Keil μVision). Le lezioni coprono tecniche avanzate di programmazione in C e Assembly (cross-compilation), oltre alla gestione di interrupt, timer, GPIO e vari dispositivi di I/O (LED, pulsanti, display, speaker, ecc.).

Come è strutturato l'esame

L'esame finale si svolge in presenza presso i laboratori LABINF ed è costituito da una prova pratica/scritta e da una prova orale opzionale. La prova al computer è articolata in due parti, le quali devono essere superate singolarmente nel medesimo appello (il non superamento di una sola parte obbliga a ripetere l'intero esame).

- **Parte 1 (Esercizi e Teoria sull'Architettura):** Questa fase consiste in domande aperte ed esercizi pratici da svolgere a mano su foglio, riguardanti gli argomenti delle lezioni, come la valutazione dei costi hardware, le tempistiche e l'ottimizzazione del codice Assembly per architetture moderne. Ha una durata di 1 ora e non è consentito consultare appunti o libri di testo. È necessario ottenere un voto sufficiente ($\geq 18/30$) in questa sezione per poter accedere alla valutazione della seconda parte.
- **Parte 2 (Programmazione su scheda ARM):** Questa prova richiede di scrivere al PC un programma funzionante in linguaggio C e Assembly ARM per la board

di riferimento. Il codice deve includere la gestione di temporizzazioni, interrupt e funzionalità di output. Durante questa parte è consentito consultare libri, appunti e materiale didattico, ma il progetto verrà valutato solo se la Parte 1 è sufficiente e se il codice compila senza errori.

- **Valutazione e Punteggio:** Il voto della prova scritta è calcolato come media matematica delle due parti, a patto che entrambe raggiungano la sufficienza ($\geq 18/30$). Il punteggio massimo è di 30/30; se il punteggio totale raggiunge o supera il 32.5, viene assegnata la lode.

- **Punti Extra (Progetto Finale):** Gli studenti che consegnano in orario le soluzioni di almeno 9 laboratori su 10 si qualificano per sviluppare un progetto finale (assegnato nelle ultime due settimane di laboratorio). Questo progetto può portare fino a 4 punti extra, che vengono sommati al voto dell'esame (se positivo e ≥ 18) e hanno una validità di un anno accademico (4 sessioni).

- **Prova Orale:** Il docente mantiene sempre il diritto di procedere con un esame orale, indipendentemente dalla decisione dello studente, nel caso in cui ritenga necessario valutare la preparazione in modo più approfondito.

- **Esonero Totale (Progetto Speciale):** È prevista l'opzione di candidarsi (entro il 17 Gennaio 2027) per un progetto speciale che sostituisce interamente l'esame. Per essere idonei, bisogna frequentare il corso per la prima volta, aver consegnato tutti i laboratori (incluso il progetto finale) nei tempi prestabiliti e superare un colloquio con i professori.

Capitolo 1

Introduction to Computer Design

1.1 Sistemi Embedded e Classi di Parallelismo

Nel panorama odierno, il mercato dei calcolatori non è uniforme. Troviamo diverse tipologie di sistemi, tra cui i **sistemi embedded**, i quali sono onnipresenti (dal settore *automotive* agli elettrodomestici, fino alle missioni aerospaziali). I problemi nel mondo *embedded* vengono spesso risolti ricorrendo a una delle seguenti soluzioni:

- **Processore standard + logica custom + software custom**
- **Processore standard + software custom**
- **DSP (Digital Signal Processor) standard + software custom**

In questo contesto, i dispositivi programmabili come le **FPGA** (Field Programmable Gate Arrays) stanno giocando un ruolo sempre più rilevante, anche grazie alla loro capacità di essere riprogrammate sul campo per correggere eventuali guasti (come avviene nei satelliti).

1.1.1 Classi di Parallelismo

Per soddisfare l'enorme richiesta di prestazioni, le architetture sfruttano diverse tipologie di parallelismo:

- **Data-level Parallelism (DLP):** Molti dati vengono elaborati contemporaneamente (es. applicare lo stesso filtro a tutti i pixel di un'immagine).
- **Task-level Parallelism (TaskLP):** Diversi task o processi operano in maniera indipendente (es. le richieste indipendenti di vari utenti su un server).

Queste classi di parallelismo vengono sfruttate da diverse architetture hardware:

1. **Instruction-level Parallelism (ILP):** Sfrutta in modo modesto il DLP, eseguendo più istruzioni di un singolo flusso in parallelo (è il focus principale per i processori general-purpose, ad esempio tramite la *pipeline*).
2. **Vector Architectures e GPU:** Sfruttano massicciamente il DLP.
3. **Thread-level Parallelism (TLP):** Sfrutta sia il DLP che il TaskLP, tipico dei sistemi multicore.
4. **Request-level Parallelism (RLP):** Sfrutta il parallelismo a livello di richiesta, tipico dei cluster o dei server.

1.2 Architettura dei Calcolatori e Requisiti

Negli ultimi decenni, il design dei calcolatori ha tratto grande vantaggio dalle innovazioni architetturali. Il divario di prestazioni tra i processori odierni e quelli passati sarebbe impossibile da giustificare unicamente con il miglioramento tecnologico dei semiconduttori (Legge di Moore).

L'architettura dei calcolatori comprende tre aspetti fondamentali:

1. **Instruction Set Architecture (ISA):** L'interfaccia hardware/software (quali istruzioni la CPU può eseguire).
2. **Organization:** L'organizzazione interna (come i moduli sono interconnessi, ad esempio le gerarchie di memoria o la logica di controllo).
3. **Hardware:** L'implementazione fisica e la tecnologia manifatturiera.

Il progettista (*computer architect*) deve bilanciare cinque vincoli fondamentali: **Requisiti funzionali, Prezzo, Potenza (Power), Prestazioni (Performance) e Affidabilità (Dependability)**.

1.2.1 La Legge di Moore

Definizione 1.1 (Legge di Moore)**.** Il numero di dispositivi (ovvero di transistor) che possono essere integrati in un singolo chip raddoppia ogni 18-24 mesi.

Nonostante le difficoltà fisiche nel scendere sotto la soglia dei nanometri, le aziende continuano a innovare, per esempio esplorando design tridimensionali, accatastando strati di silicio per mantenere alto il numero di transistor riducendo l'area bidimensionale.

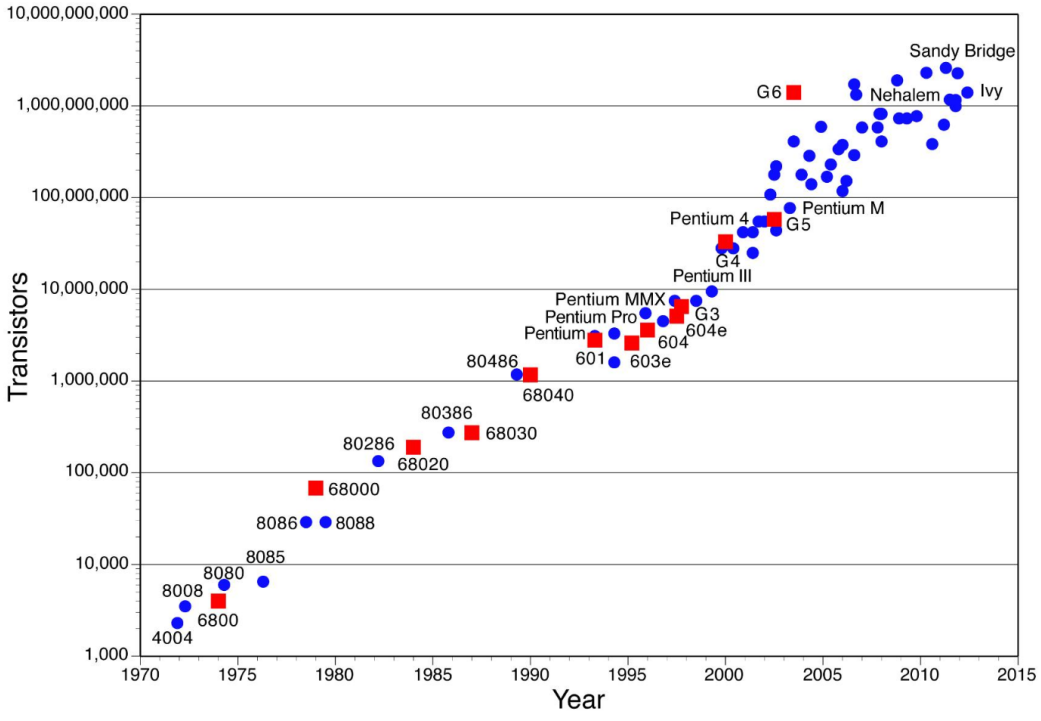


Figura 1.1: Crescita della complessità dei processori Intel nel corso degli anni.

1.3 Affidabilità (Dependability) e Sicurezza

L'affidabilità è diventata cruciale, specialmente in contesti *safety-critical* come la guida autonoma, i sistemi spaziali o semplicemente per garantire che i server cloud non producano calcoli errati. I guasti possono essere causati da bug hardware (es. il noto baco del divisore nel Pentium 4), difetti di produzione o semplice invecchiamento (usura) del silicio.

1.3.1 Metriche di Affidabilità

Per quantificare l'affidabilità si usano diverse metriche statistiche:

- **MTTF (Mean Time To Failure):** Il tempo medio previsto prima che si verifichi un guasto.
- **FIT (Failures In Time):** È il reciproco del MTTF. Viene definito come un guasto ogni miliardo di ore: 1 FIT $= \frac{1}{10^9 \text{ ore}}$.
- **MTTR (Mean Time To Repair):** Il tempo medio necessario per riparare il sistema una volta avvenuto il guasto (es. riconfigurando una FPGA in un satellite da remoto).
- **MTBF (Mean Time Between Failures):** Il tempo medio tra due guasti successivi, calcolato come:

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

L'**Availability** (Disponibilità) è definita come la probabilità che un sistema funzioni correttamente in un istante di tempo generico.

Osservazione 1.1 (Perché 1 FIT non è trascurabile?). Un valore di 1 FIT (un guasto ogni miliardo di ore) sembra eccellente se riferito a una singola persona. Tuttavia, su larga scala, il problema emerge chiaramente. Se in Europa circolano centinaia di milioni di auto, e ciascuna possiede decine di dispositivi elettronici, la somma totale delle ore operative è gigantesca. Statisticamente, questo significa avere un dispositivo che fallisce ogni 2-5 ore nell'intero parco auto europeo. Per questo motivo, industrie come STMicroelectronics puntano allo **Zero FIT**.

1.3.2 La Curva a Vasca (Bathtub Curve) e il Burn-in

Il ciclo di vita dei guasti hardware segue tipicamente una "Bathtub curve" (curva a vasca da bagno):

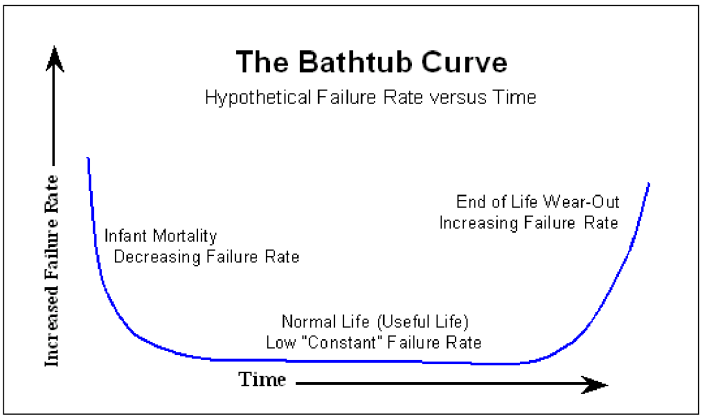


Figura 1.2: Curva a vasca (Bathtub Curve) del ciclo di vita dei guasti hardware.

1. **Mortalità infantile:** Subito dopo la fabbricazione, il tasso di guasti è altissimo a causa dei difetti di produzione non rilevati (yield basso).
2. **Vita utile:** Il tasso si abbassa e rimane stabile per molti anni (10-15 anni).
3. **Invecchiamento (Wear-out):** Alla fine del ciclo vitale, l'usura fisica del silicio fa rialzare bruscamente la curva dei guasti.

Per evitare di vendere dispositivi soggetti a "mortalità infantile", i produttori utilizzano la tecnica del **Burn-in test**. I chip appena prodotti vengono inseriti in forni industriali ad alte temperature ed eseguiti sotto forte stress di calcolo per alcune ore. Questo processo simula anni di vita operativa; i chip che sopravvivono al *burn-in* sono considerati stabili ed entrano direttamente nella fase di "vita utile".

1.4 Valutazione delle Prestazioni (Performance Evaluation)

Le prestazioni possono essere definite in modi diversi in base a chi osserva:

- L'**utente** valuta il **Response Time** (o tempo di esecuzione): il tempo trascorso dalla richiesta di esecuzione fino alla sua conclusione.
- Il **manager/amministratore** valuta il **Throughput**: la quantità totale di lavoro (es. istruzioni al secondo) completata in una data unità di tempo.

1.4.1 I Benchmark

Il tempo di esecuzione andrebbe misurato idealmente con il *workload* (carico di lavoro) reale dell'utente. Poiché questo varia da utente a utente, l'industria adotta i **Benchmark**, insiemi di programmi appositamente selezionati per simulare casi d'uso reali. Esistono diverse categorie:

- **Real programs:** Programmi veri e propri (compilatori C, processori di testo).
- **Kernels:** Piccoli frammenti di codice molto intensivi (es. Linpack).
- **Toy benchmarks:** Algoritmi classici come Quicksort o il Crivello di Eratostene.
- **Synthetic benchmarks:** Codice creato artificialmente per stressare componenti specifiche (es. Whetstone).

Tra i più noti ricordiamo le suite dello **SPEC** (Standard Performance Evaluation Corporation) per processori general purpose, e **MiBench** per il mondo dei dispositivi embedded, che raccoglie programmi specifici per ambiti come Automotive, Network e Sicurezza.

Osservazione 1.2 (La Riproducibilità). Qualsiasi misurazione tramite benchmark deve garantire la **riproducibilità**. Chi testa deve dichiarare esplicitamente: le caratteristiche dell'hardware (configurazione del sistema), il software (Sistema Operativo e Compilatore usato) e i precisi dati in input passati al programma (es. dimensione della matrice per il Quicksort), altrimenti i risultati sono inaffidabili.

1.4.2 Metriche di Confronto: Media Aritmetica e Pesata

Per confrontare le prestazioni di un processore X rispetto a un processore Y (spesso normalizzati rispetto a una macchina di riferimento), usiamo diverse tipologie di medie applicate sui tempi di esecuzione di un insieme di n benchmark.

La **Media Aritmetica** assume che tutti i benchmark siano eseguiti con la stessa frequenza:

$$\text{Arithmetic Mean} = \frac{1}{n}\sum_{i=1}^{n} \text{Time}_i$$

Tuttavia, nella realtà, alcune applicazioni sono molto più usate di altre. È quindi più corretto utilizzare la **Media Aritmetica Pesata** (Weighted arithmetic mean), assegnando un peso (Weight_i) a ciascun programma, dove la somma dei pesi è uguale a 1.

$$\text{Weighted execution time} = \sum_{i=1}^{n} (\text{Weight}_i \times \text{Time}_i)$$

Esempio 1.1 (Impatto dei Pesi sulle Prestazioni)**.** Consideriamo due programmi ($P1$ e $P2$), tre macchine (A, B, C) e tre set di pesi (W_1, W_2, W_3).

- $P1$: La macchina A impiega 1s, B impiega 10s, C impiega 20s.
- $P2$: La macchina A impiega 1000s, B impiega 100s, C impiega 20s.

Consideriamo i tempi di esecuzione dei due programmi sulle tre macchine:

	$P1$	$P2$
A	1 s	1000 s
B	10 s	100 s
C	20 s	20 s

Caso W_1: pesi 0.50 e 0.50. In questo caso i due programmi hanno la stessa importanza. Il tempo medio pesato di ciascuna macchina è:

$$\begin{aligned} T_A &= 0.50 \cdot 1 + 0.50 \cdot 1000 = 500.5\,\text{s}, \\ T_B &= 0.50 \cdot 10 + 0.50 \cdot 100 = 55\,\text{s}, \\ T_C &= 0.50 \cdot 20 + 0.50 \cdot 20 = 20\,\text{s}. \end{aligned}$$

Quindi la macchina **C** è la migliore, perché ha il tempo medio pesato più basso. Inoltre, è molto bilanciata: impiega 20 s per entrambi i programmi.

Caso W_3: pesi 0.999 e 0.001. Ora quasi tutta l'importanza è assegnata a $P1$, mentre $P2$ conta soltanto per lo 0.1%. I nuovi tempi medi pesati sono:

$$\begin{aligned} T_A &= 0.999 \cdot 1 + 0.001 \cdot 1000 = 1.999\,\text{s} \approx 2\,\text{s}, \\ T_B &= 0.999 \cdot 10 + 0.001 \cdot 100 = 10.09\,\text{s}, \\ T_C &= 0.999 \cdot 20 + 0.001 \cdot 20 = 20\,\text{s}. \end{aligned}$$

In questo caso la macchina **A** è la migliore, perché è molto veloce su $P1$, il programma che pesa quasi interamente sul risultato.

Il risultato cambia perché la macchina migliore dipende dal carico di lavoro considerato, cioè dalla frequenza con cui vengono eseguiti i programmi. Con W_1, i due programmi contano allo stesso modo e vince la macchina C, più bilanciata. Con W_3, conta quasi solo $P1$ e vince la macchina A. Non esiste quindi una macchina "migliore in assoluto": la scelta dipende dai pesi assegnati ai programmi.

1.5 Miglioramenti Architetturali

Quando si ha l'opportunità di riprogettare una componente hardware per aumentare le prestazioni, è necessario poter valutare analiticamente la bontà della scelta. Due strumenti sono essenziali: la Legge di Amdahl e l'Equazione delle Prestazioni della CPU.

1.5.1 La Legge di Amdahl

La **Legge di Amdahl** calcola lo *Speedup* (incremento di velocità) globale ottenibile migliorando solo una specifica frazione del sistema.

$$\text{Speedup} = \frac{\text{Tempo originale}}{\text{Tempo migliorato}}$$

Definiamo due parametri:

- F (Frazione migliorata): la porzione del tempo di esecuzione originale che beneficerà del miglioramento.
- S (Speedup della frazione): di quanto ho reso più veloce quella specifica frazione.

Il nuovo tempo di esecuzione sarà composto dal tempo per la porzione non migliorata $(1 - F)$ più il tempo per la porzione migliorata, ora diviso per lo speedup $\frac{F}{S}$. La Legge di Amdahl è formulata come:

$$\text{Speedup Globale} = \frac{1}{(1 - F) + \frac{F}{S}} \tag{1.1}$$

Esempio 1.2 (Valutazione di due alternative con Amdahl)**.** Siamo di fronte a due possibili scelte di progettazione, ma per ragioni di budget possiamo implementarne solo una:

1. **Miglioramento Radice Quadrata:** Velocizza le operazioni di radice quadrata di un fattore 10 ($S = 10$). Queste operazioni rappresentano il 20% del tempo del programma ($F = 0.20$).
2. **Miglioramento Floating Point globale:** Velocizza **tutte** le operazioni floating point di un fattore 2 ($S = 2$). Queste operazioni rappresentano il 50% del programma ($F = 0.50$).

Calcoliamo lo Speedup nei due casi:

$$\text{Speedup}_1 = \frac{1}{(1 - 0.20) + \frac{0.20}{10}} = \frac{1}{0.80 + 0.02} = \frac{1}{0.82} \approx 1.22$$

$$\text{Speedup}_2 = \frac{1}{(1 - 0.50) + \frac{0.50}{2}} = \frac{1}{0.50 + 0.25} = \frac{1}{0.75} \approx 1.33$$

Nonostante l'accelerazione nominale del primo modulo fosse enorme (10 volte più veloce), conviene implementare il secondo modulo (Speedup globale 1.33 contro 1.22), perché ha un impatto maggiore sull'esecuzione complessiva del programma.

1.5.2 Equazione delle prestazioni della CPU

In fase di simulazione e progettazione teorica, il tempo di CPU può essere approssimato dalla seguente equazione fondamentale:

$$\text{CPU Time} = \left(\sum_{i=1}^{n} \text{CPI}_i \times \text{IC}_i \right) \times \text{Clock cycle time} \tag{1.2}$$

Dove:

- **$\mathbf{CPI_i}$ (Cycles Per Instruction):** Cicli di clock necessari per eseguire un'istruzione di tipo i.
- **$\mathbf{IC_i}$ (Instruction Count):** Numero di istruzioni di quel tipo effettivamente eseguite dal programma.
- **Clock cycle time:** Periodo del clock (inverso della frequenza, $T = \frac{1}{f}$).

Osservazione 1.3 (Limitazioni dell'Equazione). Nelle architetture moderne basate su *pipeline*, calcolare analiticamente il CPI puro è molto difficile e impreciso. Il numero reale di cicli di clock richiesti da un'istruzione dipenderà dinamicamente da:

- Dipendenze tra istruzioni contigue (es. stalli della pipeline).
- Comportamento della gerarchia di memoria (es. una *cache miss* fermerà l'istruzione in attesa della memoria RAM centrale, variando enormemente i cicli richiesti).

Per ottenere dati affidabili è dunque necessario ricorrere a complessi simulatori software.

Capitolo 2

Instruction Set Principles

2.1 Introduzione all'Instruction Set Architecture (ISA)

L'**Instruction Set Architecture (ISA)** rappresenta l'interfaccia tra l'hardware e il software; è il modo in cui il calcolatore viene visto dal programmatore o dal compilatore. Il progettista di un'architettura ha a disposizione molte alternative, e ogni scelta modifica il modo in cui le istruzioni vengono codificate.

Le diverse alternative vengono valutate in base a:

- Prestazioni del processore (Processor performance);
- Complessità del processore (Processor complexity);
- Complessità del compilatore (Compiler complexity);
- Dimensione del codice generato (Code size);
- Consumo di potenza (Power consumption).

Osservazione 2.1. La scelta dell'ISA è critica. Ad esempio, oggi molti processori per laptop (come gli Intel 8086 e successivi) si presentano all'esterno come architetture CISC (Complex Instruction Set Computer) con istruzioni molto complesse, ma internamente decodificano queste istruzioni in micro-operazioni tipiche di un'architettura RISC (Reduced Instruction Set Computer) per ottimizzare le prestazioni.

2.2 Modelli di Memorizzazione Interna

Una delle prime scelte architetturali riguarda dove salvare i dati che il processore andrà a elaborare. Esistono diverse opzioni storiche e attuali:

1. **Stack**: I dati sono salvati in una struttura a pila gestita dal *Top of Stack (TOS)*. Le operazioni aritmetiche non specificano gli operandi, ma utilizzano implicitamente i primi elementi dello stack.
2. **Accumulatore**: Utilizzato in architetture storiche (e oggi in moduli di accelerazione AI come i MAC - Multiply and Accumulate). L'accumulatore funge sempre da operando implicito e da registro di destinazione.
3. **Registro - Memoria (CISC)**: Le operazioni possono avvenire combinando un dato presente in un registro e uno presente direttamente in memoria (es. architetture 80x86).

4. **Registro - Registro / Load-Store (RISC)**: Tutte le operazioni aritmetico-logiche avvengono esclusivamente tra registri interni. Per elaborare un dato in memoria, esso deve essere prima caricato in un registro tramite un'istruzione indipendente. (Es. RISC-V, ARM, MIPS).

Esempio 2.1. Consideriamo l'operazione di alto livello $C = A + B$. Vediamo come viene tradotta nei diversi modelli:

- **Stack**: `push A`, `push B`, `add`, `pop C`. (L'istruzione `add` sa già dove prendere i dati).
- **Accumulatore**: `load A`, `add B`, `store C`. (`add` somma B al valore già in accumulatore).
- **Registro-Memoria**: `load R1, A`, `add R1, B`, `store R1, C`.
- **Registro-Registro (Load-Store)**: `load R1, A`, `load R2, B`, `add R3, R1, R2`, `store R3, C`.

Nonostante l'approccio Load-Store richieda più istruzioni, le singole istruzioni sono molto più semplici, permettendo di abbassare il consumo di potenza e aumentare le prestazioni riducendo il tempo di ciclo (eseguendo idealmente un'istruzione per ciclo di clock).

2.3 Organizzazione della Memoria e Accesso ai Dati

L'accesso alla memoria solleva il problema di come i dati (composti da più byte) vengano salvati. Esistono due convenzioni principali:

- **Little Endian**: Il byte meno significativo del dato si trova all'indirizzo più basso. (Convenzione che useremo maggiormente).
- **Big Endian**: Il byte più significativo si trova all'indirizzo più basso.

Il processore e la memoria devono condividere la stessa convenzione, altrimenti i valori letti saranno errati.

2.4 Modi di Indirizzamento (Addressing Modes)

I modi di indirizzamento definiscono come il processore calcola l'*Effective Address* (indirizzo effettivo) per accedere ai dati. Sperimentalmente, **Displacement, Immediato e Registro Indiretto** coprono dal 75% al 99% dei casi d'uso nei benchmark.

- **Register mode** (`Add R4, R3`): Nessun accesso a memoria. Il dato è già nel banco dei registri (General Purpose Registers). Altamente utilizzato.
- **Immediate mode** (`Add R4, #3`): Il dato è una costante definita direttamente all'interno dell'istruzione. Analisi empiriche mostrano che 8-16 bit sono sufficienti per rappresentare il 50-80% dei valori immediati necessari.
- **Displacement mode** (`Add R4, 100(R1)`): L'indirizzo effettivo è la somma di una costante (es. 100) e il valore contenuto in un registro base (R1). Molto utile per accedere a variabili locali o campi di struct. Solitamente richiede dai 12 ai 16 bit nell'istruzione.
- **Register Deferred / Indirect** (`Add R4, (R1)`): Il registro contiene l'indirizzo di memoria del dato (funge da puntatore).

- **Indexed / Scaled / Auto-increment**: Modi più complessi (molto usati nei CISC, quasi assenti nei RISC per mantenere semplice l'hardware). Ad esempio l'Auto-incremento è utile per scorrere i vettori, mentre lo Scaled mode moltiplica il valore dell'indice per la dimensione del dato (es. 1, 4, 8 byte).

2.5 Operazioni nell'Instruction Set e Frequenze d'Uso

Le istruzioni si dividono in categorie logiche:

Tipo di Operatore	**Esempi**
Aritmetiche e Logiche	Add, And, Subtract, Or
Trasferimento Dati	Load, Store (su macchine con memory addressing)
Controllo	Branch (salti condizionati), Jump (salti incondizionati), Call, Return
Sistema	System call, gestione memoria virtuale
Virgola Mobile (Floating Point)	Add FP, Multiply FP

Tabella 2.1: Classificazione delle istruzioni.

Analizzando i benchmark reali (es. SPECint2006 per RISC-V e programmi interi per 80x86):

- Nelle architetture CISC (80x86) storiche si cercava di ottimizzare le istruzioni più frequenti (Load 22%, Branch 20%, Compare 16%, Store 12%).
- Nelle architetture RISC (come RISC-V), le operazioni **ALU dominano (circa 42%)** seguite da **Load (24%)**, **Branches (17%)** e **Stores (11%)**. L'obiettivo dei RISC non è ottimizzare un'istruzione specifica, ma garantire che *tutte* impieghino lo stesso tempo di esecuzione.

2.6 Istruzioni di Controllo del Flusso (Control Flow)

Le istruzioni di controllo del flusso modificano il Program Counter (PC) e si dividono in: salti condizionati (*conditional branches*), salti incondizionati (*jumps*), chiamate a procedura (*procedure calls*) e ritorni (*returns*).

2.6.1 L'indirizzo di Destinazione (Destination Address)

Normalmente, per i salti condizionati, l'indirizzo di destinazione viene specificato come un **displacement rispetto al valore corrente del Program Counter (PC-Relative)**.

Osservazione 2.2. Usare un indirizzamento relativo al PC (es. PC + offset) è fondamentale perché permette di generare codice *Position Independent* (indipendente dalla posizione di memoria in cui il programma viene caricato). Evita l'uso di indirizzi assoluti che vincolerebbero il programma a una specifica zona di memoria.

2.6.2 Salti a Registro (Register Indirect Jumps) e Procedure

Per distanze molto grandi, o per indirizzi non noti a tempo di compilazione, si carica l'indirizzo target in un registro e si esegue un salto indiretto. Questo permette di:

- Implementare costrutti come `switch` / `case` in C;

- Supportare librerie dinamiche condivise (.dll, .so);
- Supportare le funzioni virtuali (Object-Oriented Programming).

2.6.3 Valutazione delle Condizioni

I salti condizionati (di gran lunga i più eseguiti, in particolare le comparazioni "Minore di" e "Uguale a", che coprono il 70% dei casi) possono essere implementati in vari modi a seconda dell'architettura:

Metodo	Esempi	Caratteristiche
Condition Code (CC)	80x86, ARM	L'ALU setta automaticamente dei flag (Zero, Carry, Negative). Vantaggio: a volte la condizione è "gratis". Svantaggio: il CC è uno stato extra, crea dipendenze tra l'istruzione aritmetica e il salto. (In ARM i CC possono rendere *ogni* istruzione condizionale).
Condition Register	MIPS, Alpha	Testa un registro arbitrario in seguito a un confronto (es. se uguale a zero). Semplice, ma può richiedere confronti extra.
Compare and Branch	RISC-V, PA-RISC	La comparazione (es. "maggiore", "minore") è inclusa direttamente nell'istruzione di salto. Vantaggio: 1 istruzione invece di 2. Svantaggio: può creare percorsi critici lunghi (critical paths) nell'hardware del processore.

Tabella 2.2: Valutazione delle condizioni nei rami di esecuzione.

2.6.4 Gestione delle Procedure (Procedure Calls)

Quando si chiama una procedura, è necessario salvare due tipi di informazioni:

1. **Return Address**: L'indirizzo a cui ritornare (tipicamente salvato in un registro dedicato).
2. **Registri acceduti**: Devono essere preservati. Esistono due convenzioni:
   - *Caller saving*: Il chiamante salva i registri prima di invocare la procedura.
   - *Callee saving*: La procedura invocata salva i registri che intende "sporcare" e li ripristina prima del return.

2.7 Tipi e Dimensioni degli Operandi

La dimensione minima indirizzabile è tipicamente il *Byte* (8 bit). Le tipologie supportate solitamente includono:

- Char (1 byte)
- Half word (2 byte)
- Word (4 byte - standard per interi a 32 bit, copre il 74% degli accessi per programmi interi)
- Double word (8 byte - molto usata per i numeri reali Double Precision)

- Floating-point singola e doppia precisione (4 e 8 byte).

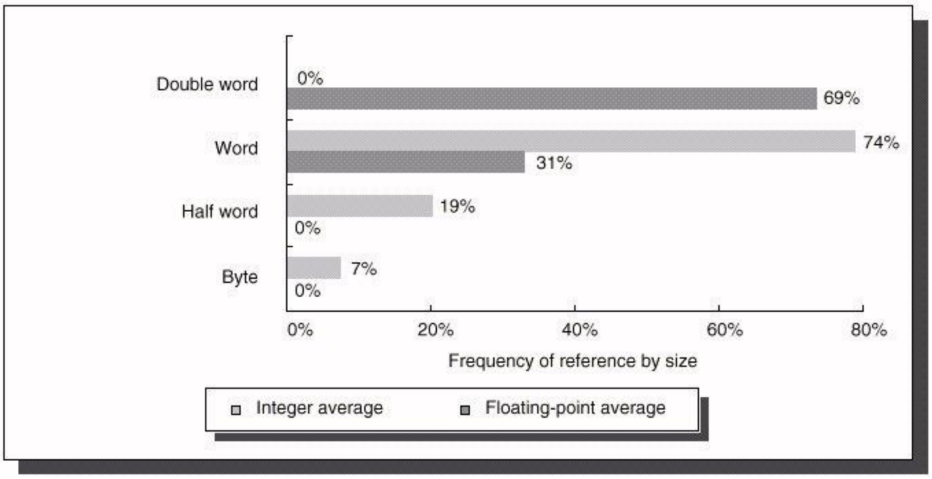


Figura 2.1: Distribuzione degli accessi ai dati per dimensione.

2.8 Codifica dell'Instruction Set (Instruction Set Encoding)

Come vengono codificate le istruzioni (Opcode, registri, costanti) nei bit a disposizione (es. 32 bit)? Dipende da quante istruzioni e da quanti modi di indirizzamento esistono. Esistono tre filosofie principali:

2.8.1 Codifica Variabile (Es. Intel 80x86, VAX)

- La lunghezza dell'istruzione varia (da 1 a molti byte).
- Può supportare un numero qualsiasi di operandi e include specificatori di indirizzamento direttamente accanto all'operando.
- **Vantaggio**: Dimensione del codice generato (Code size) minima.
- **Svantaggio**: Prestazioni ridotte, decodifica hardware estremamente complessa (il calcolo del PC successivo dipende dalla comprensione immediata della lunghezza dell'istruzione corrente).

2.8.2 Codifica Fissa (Es. RISC-V, ARM, MIPS)

- La lunghezza dell'istruzione è sempre fissa (tipicamente 32 bit).
- Numero fisso di operandi (sempre 3 per le operazioni ALU) e campi fissi.
- **Vantaggio**: Massime prestazioni e facilità di progettazione del datapath (il calcolo del PC successivo è semplicemente `PC + 4`).
- **Svantaggio**: Dimensione del codice maggiore (anche istruzioni semplici occupano 32 bit).

2.8.3 Codifica Ibrida (Es. RISC-V Compressed, ARM Thumb-2)

- Offre formati multipli decisi dall'opcode (es. istruzioni a 16 bit per operazioni molto comuni e 32 bit per il resto).
- **Vantaggio**: Ottimo compromesso (trade-off) tra dimensione del codice compatta e alte prestazioni in fase di decodifica, lasciato spesso alla gestione del compilatore.

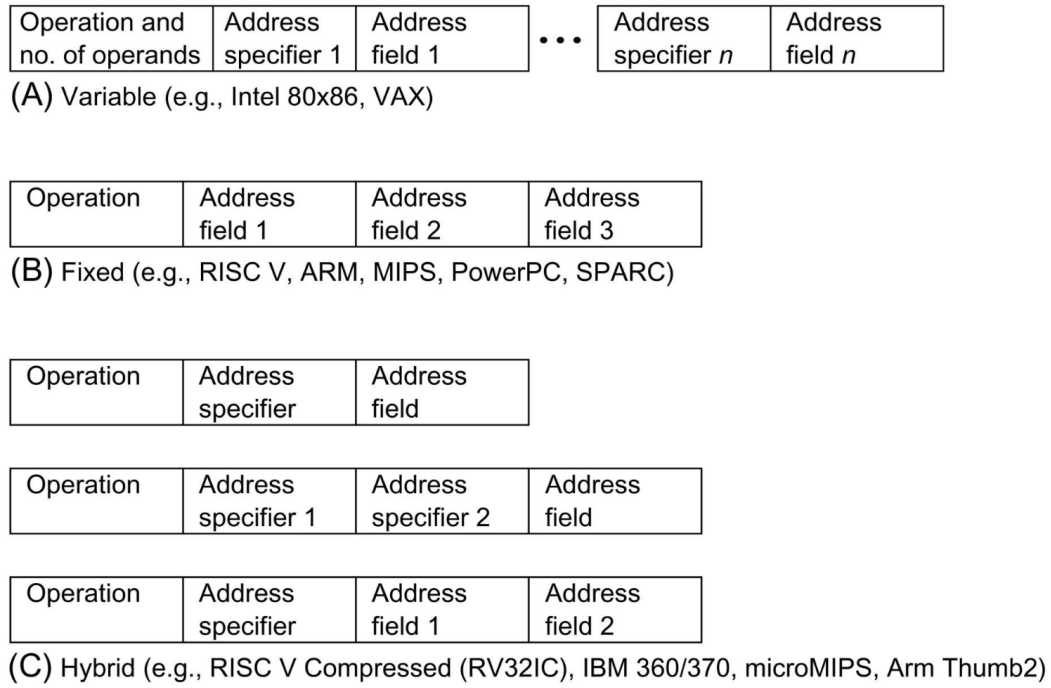


Figura 2.2: Confronto tra i modelli di decodifica: Variabile, Fissa e Ibrida.

Osservazione 2.3 (Conflitti Progettuali (Conflicting Issues)). Il progettista di ISA affronta un problema di ottimizzazione contrastante: bilanciare la compattezza del codice (*code size*), la ricchezza dell'ISA (numero di registri e modi di indirizzamento) e la semplicità/velocità dell'hardware di *fetch e decodifica*. Architetture moderne tendono verso la semplicità (RISC) affiancando estensioni compresse (Thumb2 / RV32IC) per recuperare compattezza del codice.

Capitolo 3

Introduzione a RISC-V

Osservazione 3.1. Negli anni Ottanta, John Hennessy e David Patterson svilupparono il processore MIPS, basato sull'idea di un *Instruction Set* ridotto (RISC). L'obiettivo era semplificare le istruzioni per evitare stalli nell'elaborazione. Questa architettura rispecchiava una linea di produzione con diversi "stadi" (pipeline) in cui ogni operazione richiedeva lo stesso tempo delle altre. RISC-V nasce come naturale evoluzione open-source di questi concetti.

3.1 Storia e Caratteristiche Generali

RISC-V è un'architettura ISA (*Instruction Set Architecture*) basata su standard aperto e licenza libera. Proposta originariamente nel maggio 2010 dal Parallel Computing Laboratory (Par Lab) dell'Università di Berkeley (guidato dal Prof. David Patterson), si caratterizza per il nome "V" che ha molteplici significati:

- È la **quinta versione** dell'ISA sviluppata a Berkeley.
- Il numero romano V rappresenta i **cinque stadi** della pipeline classica.
- Sta per **Variations** e **Vectors**, indicando la possibilità di aggiungere estensioni vettoriali.

Nel 2015 è nata la *RISC-V Foundation* (oggi legata anche alla *Linux Foundation*), che conta ormai centinaia di aziende e istituzioni membri.

Osservazione 3.2. Il nostro gruppo di ricerca al Politecnico lavora attivamente su implementazioni RISC-V sviluppate dall'ETH di Zurigo. Abbiamo anche esteso il processore con un acceleratore crittografico, motivo per cui nel corso vedremo alcune istruzioni orientate alla sicurezza. Inoltre, utilizzeremo un simulatore architetturale sviluppato internamente che ci permette di analizzare nel dettaglio cosa accade a livello micro-architetturale (dipendenze, stalli, ritardi), a differenza dei semplici emulatori disponibili online.

3.1.1 L'Architettura Load-Store

RISC-V implementa una pura architettura **Load-Store**.

Definizione 3.1 (Architettura Load-Store)**.** In un'architettura Load-Store, la memoria principale viene acceduta esclusivamente tramite istruzioni specifiche di caricamento (`load`) e salvataggio (`store`). Non è possibile eseguire un'operazione aritmetica o logica includendo un operando che risiede direttamente in memoria. I dati devono prima essere caricati nei registri interni del processore.

3.1.2 Tipi di Dati e Formato delle Istruzioni

Nel nostro corso ci concentreremo sulla versione base a 32 bit, denominata **RV32I** (o **RV32G** quando include estensioni standard). I tipi di dati supportati nativamente includono:

- **Byte:** 8 bit.
- **Half Word:** 16 bit.
- **Word:** 32 bit.
- **Double Word:** 64 bit (non usata nella nostra versione a 32 bit).
- Numeri *floating-point* a singola (32 bit) e doppia (64 bit) precisione.

La lunghezza delle istruzioni nel set base è **fissa a 32 bit**, il che semplifica notevolmente la fase di decodifica. Tuttavia, RISC-V permette un *variable encoding* in blocchi multipli di 16 bit per ridurre la dimensione del codice (estensione C - Compressed).

•Fixed-length 32-bit instructions for the base versions
- **32-bit instructions have their lowest two bits set to 11**
- **16-bit instructions may have their lowest two bits equal to 00, 01, or 10**

Figura 3.1: I 2 bit meno significativi indicano la lunghezza dell'istruzione: `11` identifica istruzioni a 32 bit.

Il sistema di memoria utilizzato è **Little-Endian**: l'indirizzo base di una Word o Half-Word punta al suo byte meno significativo. Le istruzioni a 32 bit devono essere rigorosamente **allineate in memoria** (indirizzi multipli di 4).

3.2 Rappresentazione dei Dati

3.2.1 Interi e Complemento a Due

L'intera aritmetica del processore per i numeri interi si basa sul **complemento a due (2C)**. Questo è fondamentale per gestire correttamente somme, sottrazioni e salti condizionati. Dato un numero binario A di N bit, il complemento a due è definito come:

$$-A = 2^N - A = \overline{A} + 1$$

Esempio 3.1. Lavorando su 3 bit, l'intervallo di rappresentazione va da -4 a $+3$. Il valore binario `100` rappresenta -4_{10} se valutato in complemento a due, ma rappresenta $+4_{10}$ se considerato *unsigned*. Le istruzioni assembler dovranno perciò differenziare le operazioni con segno da quelle senza segno (es. `slt` vs `sltu`).

Using a 3-bit representation :

+ 3_{10} → 011_{2C}

- 1_{10} → 111_{2C}

000_{2C} → + 0_{10}

100_{2C} → - 4_{10}

Binary	Decimal
000	0
001	1
010	2
011	3
100	-4
101	-3
110	-2
111	-1

Figura 3.2: Esempio di rappresentazione in complemento a due su 3 bit.

3.2.2 Floating-Point (IEEE 754)

Se si adotta l'estensione F o D, RISC-V utilizza lo standard IEEE 754. Per i 32 bit, la word è divisa in:

- **Segno:** 1 bit
- **Esponente:** 8 bit
- **Mantissa:** 23 bit

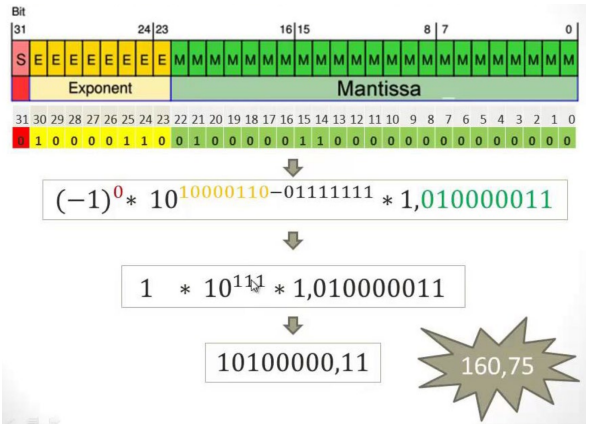


Figura 3.3: Formato a 32 bit IEEE 754 per i numeri in virgola mobile.

3.3 Organizzazione dell'Instruction Set e Registri

La modularità è il cuore di RISC-V. Un processore standard viene definito tramite sigle, ad esempio **RV64IMAFD** (spesso abbreviato in **RV64G**, *General-purpose*):

- **I:** Base integer ISA (32 registri interi).
- **M:** Moltiplicazione e divisione.
- **A:** Istruzioni atomiche.
- **F, D:** Floating-point a singola e doppia precisione.

Name of base or extension	Functionality
RV32I	Base 32-bit integer instruction set with 32 registers
RV32E	Base 32-bit instruction set but with only 16 registers; intended for very low-end embedded applications
RV64I	Base 64-bit instruction set; all registers are 64-bits, and instructions to move 64-bit from/to the registers (LD and SD) are added
M	Adds integer multiply and divide instructions
A	Adds atomic instructions needed for concurrent processing; see Chapter 5
F	Adds single precision (32-bit) IEEE floating point, includes 32 32-bit floating point registers, instructions to load and store those registers and operate on them
D	Extends floating point to double precision, 64-bit, making the registers 64-bits, adding instructions to load, store, and operate on the registers
Q	Further extends floating point to add support for quad precision, adding 128-bit operations
L	Adds support for 64- and 128-bit decimal floating point for the IEEE standard
C	Defines a compressed version of the instruction set intended for small-memory-sized embedded applications. Defines 16-bit versions of common RV32I instructions
V	A future extension to support vector operations (see Chapter 4)
B	A future extension to support operations on bit fields
T	A future extension to support transactional memory
P	An extension to support packed SIMD instructions: see Chapter 4
RV128I	A future base instruction set providing a 128-bit address space

Figura 3.4: Principali estensioni dell'Instruction Set RISC-V.

3.3.1 I Registri

La versione a 32 bit (RV32I) possiede **32 registri generici interi** da 32 bit, numerati da `x0` a `x31`, più il Program Counter (`PC`).

- **Il registro `x0` (zero):** È hardwired a 0. Non può essere modificato e restituisce sempre 0. È estremamente utile per sintetizzare altre operazioni (es. copiare registri o forzare zeri).
- **Registri Floating Point:** Se l'estensione è attiva, si aggiungono 32 registri `f0` - `f31`.

Osservazione 3.3. Lo standard ABI (*Application Binary Interface*) definisce nomi specifici per i registri in base al loro ruolo nelle funzioni C/C++ (es. `ra` per `x1` come Return Address, `sp` per `x2` come Stack Pointer, `a0-a7` per gli argomenti). Tuttavia, in questa fase introduttiva in Assembly puro, **utilizzeremo la notazione nativa** (`x0, x1, ...`) e non ci preoccuperemo delle convenzioni di chiamata delle funzioni.

Register	ABI Name	Description	Register	ABI Name	Description
x0	zero	hardwired zero	x16	a6	function argument 6
x1	ra	return address	x17	a7	function argument 7
x2	sp	stack pointer	x18	s2	saved register 2
x3	gp	global pointer	x19	s3	saved register 3
x4	tp	thread pointer	x20	s4	saved register 4
x5	t0	temporary register 0	x21	s5	saved register 5
x6	t1	temporary register 1	x22	s6	saved register 6
x7	t2	temporary register 2	x23	s7	saved register 7
x8	s0 / fp	saved register 0 / frame pointer	x24	s8	saved register 8
x9	s1	saved register 1	x25	s9	saved register 9
x10	a0	function argument 0 / return value 0	x26	s10	saved register 10
x11	a1	function argument 1 / return value 1	x27	s11	saved register 11
x12	a2	function argument 2	x28	t3	temporary register 3
x13	a3	function argument 3	x29	t4	temporary register 4
x14	a4	function argument 4	x30	t5	temporary register 5
x15	a5	function argument 5	x31	t6	temporary register 6

Figura 3.5: Banco registri RV32I con le relative convenzioni ABI.

3.4 Modi di Indirizzamento

L'architettura supporta tre modi principali per l'indirizzamento di dati in memoria e istruzioni:

1. **Immediate (Immediato):** Il dato è contenuto direttamente nell'istruzione (tipicamente 12 bit con estensione del segno).

$$\texttt{addi x1, x2, 32} \quad \Rightarrow \quad x1 \leftarrow x2 + 32$$

$$\texttt{addi x1, x0, 32} \quad \Rightarrow \quad x1 \leftarrow 32$$

2. **Displacement (Spiazzamento):** Utilizza una base salvata in un registro più un offset (12 bit) costante.

$$\texttt{lw x1, 30(x2)} \quad \Rightarrow \quad x1 \leftarrow \text{MEM}[x2 + 30]$$

3. **Register Indirect (Indiretto a registro):** Non esiste come modalità nativa, ma è un caso particolare del Displacement con offset nullo.

$$\texttt{lw x1, 0(x2)} \quad \Rightarrow \quad x1 \leftarrow \text{MEM}[x2]$$

Tramite l'uso di `x0`, si può simulare anche un **indirizzamento assoluto** in un intervallo ristretto (i primi 4 KB di memoria), scrivendo `lw x1, 30(x0)`.

3.5 Formato delle Istruzioni

Le istruzioni a 32 bit sono suddivise in categorie di codifica. Tutte mantengono l'`Opcode` nei 7 bit meno significativi. I formati principali sono:

- **R-Type (Register):** Operazioni tra registri (es. somma). Struttura: `Opcode(7)`, `rd(5)`, `func3(3)`, `rs1(5)`, `rs2(5)`, `func7(7)`.
- **I-Type (Immediate):** Operazioni con immediati o Load. L'immediato a 12 bit sostituisce `rs2` e `func7`.

- **S-Type (Store):** Utilizzato per salvare in memoria. Il campo immediato a 12 bit è "spezzato" (`Imm[11:5]` e `Imm[4:0]`) per mantenere il registro sorgente `rs2` nella stessa posizione delle R-Type, semplificando l'hardware decodificatore.
- **U-Type (Upper Immediate):** Istruzioni con immediati estesi a 20 bit (es. `LUI`, `AUIPC`), usate per gestire grandi costanti e indirizzamenti a lungo raggio.

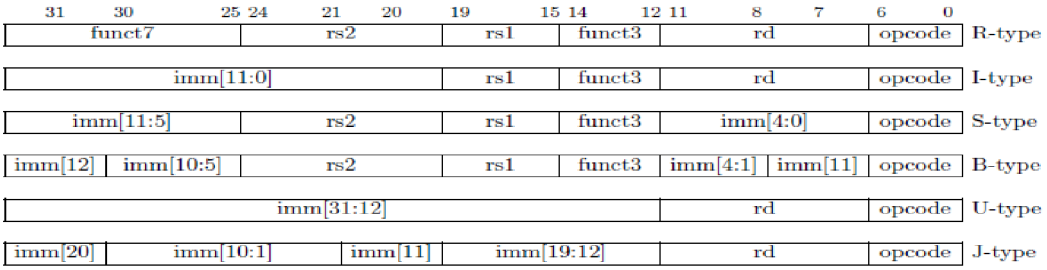


Figura 3.6: Struttura dei formati R, I, S, U dell'instruction set RISC-V.

3.6 Analisi delle Istruzioni Base

3.6.1 Load e Store

Gestiscono il trasferimento dati memoria-registri. Nella sintassi, la Load ha il registro di destinazione come primo operando, mentre la Store ha il registro sorgente come primo operando.

- `lw x1, 28(x8)`: Carica una word (32 bit).
- `lb x1, 28(x8)`: Carica un byte con **estensione del segno**.
- `lbu x1, 28(x8)`: Carica un byte (*unsigned*) con estensione di zeri.
- `sw x2, 28(x8)`: Salva una word.
- `sb x2, 28(x8)`: Salva un solo byte.

Osservazione 3.4 (L'importanza dell'estensione del segno). Quando carichiamo un byte (8 bit) in un registro a 32 bit (`lb`), dobbiamo replicare il bit più significativo (il segno) nei restanti 24 bit a sinistra. Questo serve a preservare il valore matematico del numero in complemento a due (es. -3 su 8 bit deve rimanere -3 su 32 bit). Se usiamo `lbu`, i 24 bit aggiuntivi vengono forzati a zero.

Caricamento di indirizzi e pseudo-istruzione LA

Poiché l'immediato del displacement è limitato a 12 bit (da -2048 a +2047), indirizzare variabili arbitrarie in memoria richiede un trucco. Il compilatore offre la pseudo-istruzione `la` (*Load Address*):

```
la x5, var
```

In realtà, questa viene tradotta in due istruzioni reali che sfruttano il PC e un immediato superiore:

1. `auipc x5, <upper_20_bits>`: Aggiunge i 20 bit alti (shiftati a sinistra) al Program Counter e salva in `x5`.
2. `addi x5, x5, <lower_12_bits>`: Aggiusta i 12 bit inferiori per centrare l'indirizzo esatto.

3.6.2 Operazioni Aritmetico-Logiche (ALU)

L'ALU esegue istruzioni su registri o tra registri e immediati (es. `add`, `addi`, `sub`, `and`, `sll`). Il registro `x0` è vitale per sintetizzare istruzioni "mancanti" nel set:

Esempio 3.2. Caricare una costante in un registro:

`addi x1, x0, 10` (Esegue $x1 \leftarrow 0 + 10$)

Copiare il contenuto di un registro (Move):

`add x1, x0, x2` (Esegue $x1 \leftarrow 0 + x2$)

No-Operation (NOP):

`addi x0, x0, 0` (Aggiunge 0 a 0 e salva nel cestino $x0$)

3.6.3 Salti e Diramazioni (Branches e Jumps)

RISC-V utilizza istruzioni di confronto e salto integrate (senza usare *Condition Codes/Flags* separati come in altre ISA).

Salti Incondizionati

Le istruzioni `jal` (*Jump And Link*) e `jalr` saltano a un indirizzo e salvano l'indirizzo di ritorno nel registro di destinazione.

- **Jump semplice:** `j label` viene tradotto come `jal x0, label` (salta e scarta il ritorno in `x0`).
- **Chiamata a funzione:** `call label` salva il ritorno di default in `x1` (`ra`).
- **Ritorno:** `ret` viene tradotto come `jalr x0, 0(x1)` (salta all'indirizzo in `x1`).

Salti Condizionati (Branches)

Le istruzioni confrontano direttamente due registri (es. `beq`, `bne`, `blt`, `bge`). L'immediato per il target del salto è a 12 bit, limitando il raggio del salto a ± 4 KB dal PC attuale. Anche qui, l'uso di `x0` permette pseudo-istruzioni eleganti:

- `beqz x3, label` $\rightarrow$ Tradotta come `beq x3, x0, label` (Salta a label se il registro x3 è uguale a zero.)
- `bgtz x3, label` $\rightarrow$ Tradotta come `blt x0, x3, label` (salta se $0 < x3$).

Osservazione 3.5 (Testare lo Zero in modo asimmetrico). Per generare un valore booleano basato su una condizione, possiamo usare `slt` (*Set Less Than*). Ad esempio, se vogliamo settare `x1 = 1` se `x2 == 0`, non esiste un'istruzione nativa `seqz`. Il compilatore utilizzerà:

```
sltiu x1, x2, 1
```

Ovvero: Setta `x1` a 1 se `x2` è strettamente minore di 1 valutato senza segno (l'unico intero *unsigned* minore di 1 è lo 0).

Capitolo 4

Introduzione a gem5 e RISC-V

4.1 Il Simulatore gem5

Durante il corso utilizzeremo **gem5**, un simulatore architetturale e micro-architetturale open source. Lo scopo principale di questo strumento è permetterci di valutare come specifiche scelte micro-architetturali impattino le prestazioni complessive di un sistema.

Definizione 4.1 (Simulatore gem5)**.** gem5 è un simulatore di livello sistema e micro-architetturale che supporta varie architetture ISA (Instruction Set Architecture), tra cui ARM, x86 e RISC-V. Permette di simulare un intero sistema offrendo un controllo a grana fine su tutti i suoi componenti (sistema di cache, branch prediction, unità funzionali della CPU).

Nella nostra configurazione, utilizzeremo gem5 per eseguire programmi scritti in **assembly RISC-V**. Non faremo girare un sistema operativo completo (configurazione *bare metal*); il processore eseguirà direttamente il nostro codice.

Il simulatore è estremamente flessibile: è scritto principalmente in C++ (per i moduli eseguibili) ed è configurabile tramite script Python. Modificando i parametri negli script Python, potremo cambiare le caratteristiche delle unità funzionali (es. le latenze) e osservare come variano i tempi di esecuzione.

4.1.1 La Pipeline a 5 Stadi

L'architettura CPU che utilizzeremo in gem5 è stata sviluppata specificamente per questo corso e rispecchia fedelmente la classica **pipeline a 5 stadi** descritta da Hennessy e Patterson:

1. **Fetch** (F): Prelevamento dell'istruzione dalla memoria.
2. **Decode** (D): Decodifica dell'istruzione e lettura dei registri.
3. **Execute** (E): Esecuzione dell'operazione nell'ALU.
4. **Memory** (M): Accesso alla memoria dati (se necessario).
5. **Writeback** (W): Scrittura del risultato nel file dei registri.

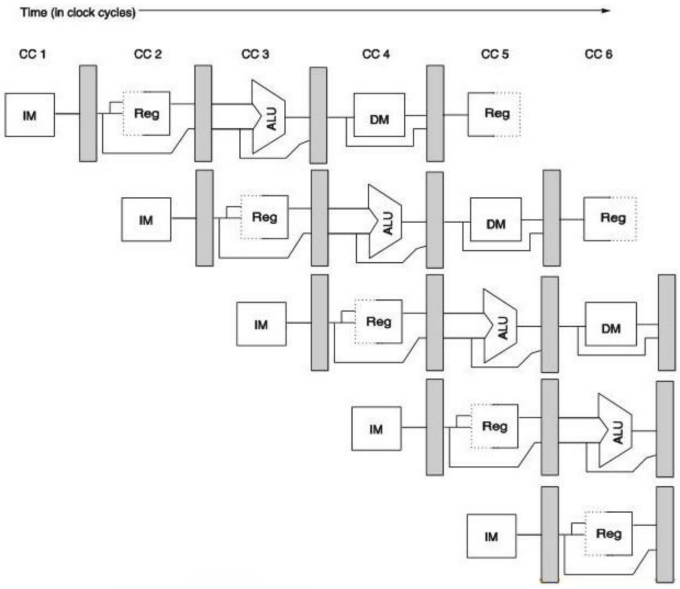


Figura 4.1: Rappresentazione dei 5 stadi della pipeline RISC-V.

4.2 Struttura dei Programmi in Assembly

L'architettura di riferimento implementa il modello **Load/Store** e si basa su un'architettura **Harvard**, in cui la memoria dati e la memoria istruzioni sono separate e collegate esternamente al processore. Questo è fondamentale per le prestazioni, in quanto permette alla CPU di accedere simultaneamente a entrambe le memorie nello stesso ciclo di clock.

Coerentemente con questa divisione fisica, anche a livello logico i nostri programmi in assembly (tipicamente salvati con estensione `.s`) saranno divisi in due sezioni principali tramite specifiche direttive per il compilatore:

- `.data` (o `.section .data`): È la sezione dedicata ai dati. Qui vengono allocate le variabili, i vettori e le costanti che risiederanno nella memoria dati.
- `.text` (o `.code` / `.section .text`): È la sezione che contiene il codice del programma vero e proprio (routine, sottoroutine, funzioni) che risiederà nella memoria istruzioni.

Osservazione 4.1 (L'importanza dei commenti e dell'indentazione). Le istruzioni assembly sono talmente elementari che il loro scopo di alto livello (es. moltiplicare per 8 tramite uno shift) si perde facilmente. Il professore sottolinea che **commentare il codice** (es. spiegare cosa si sta facendo a livello logico, mappare i registri alle variabili) è obbligatorio, specialmente in sede d'esame. Inoltre, le istruzioni vanno sempre indentate (con un tab o 4 spazi) lasciando le etichette (labels) a margine sinistro per garantirne la leggibilità.

4.3 Confronto Prestazionale: RISC vs CISC

Per capire l'impatto del set di istruzioni sull'architettura, analizziamo un semplice frammento di codice che esegue l'operazione $C = A + B$ in memoria, confrontando un approccio RISC (RISC-V) e un approccio CISC (Intel 8086).

Esempio 4.1 (Somma di due variabili in memoria)**.** In RISC-V, non è possibile sommare direttamente un elemento di memoria con un registro. Bisogna prima caricare i dati nei registri (Load), eseguire la somma, e salvare il risultato (Store).

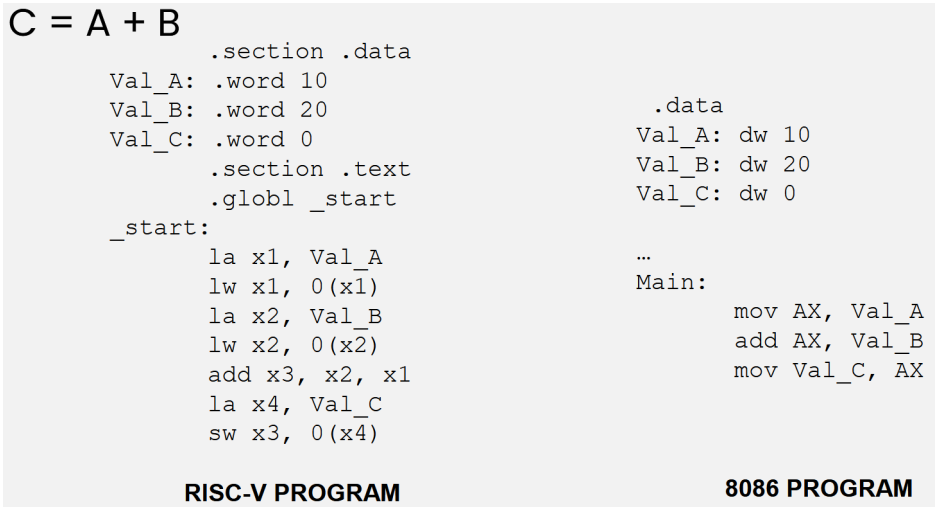


Figura 4.2: Confronto tra RISC-V (Load/Store) e 8086 (CISC) per l'operazione $C = A + B$.

Nel 8086 (CISC), un'istruzione può includere sia l'operazione aritmetica sia l'accesso alla memoria:

Metrica	**RISC-V (Load/Store)**	**8086 (CISC)**
Numero di istruzioni	7 (10 con espansione pseudo-istr.)	3
Dimensione codice	40 byte	8 byte
Tempo di esecuzione	**15 Cicli di Clock**	**33 Cicli di Clock**

Tabella 4.1: Confronto tra approccio RISC e CISC per l'operazione $C = A + B$.

Osservazione 4.2 (Analisi del confronto). Anche se il programma CISC è molto più compatto (meno istruzioni e meno byte), l'architettura RISC impiega meno cicli di clock per terminare l'elaborazione. Nel CISC, infatti, la complessa gestione interna necessaria per accedere alla memoria e fare calcoli nella stessa istruzione rallenta notevolmente l'esecuzione complessiva. Oggi, la maggior parte dei processori CISC traduce internamente le istruzioni complesse in micro-istruzioni di tipo RISC.

4.4 Ambiente di Sviluppo e Simulazione

Per sviluppare e testare il codice, il ciclo di lavoro prevede tre fasi:

1. **Compilazione:** Si utilizza la toolchain `gcc` per RISC-V (comando `riscv_compile`).
2. **Simulazione:** Si esegue il simulatore passandogli lo script Python di configurazione, l'eseguibile e il nome del file di log di output (comando `gem5_run gem5_config.py program program.log`).
3. **Visualizzazione:** Tramite **ASE Studio** (o `gem5_pipeline_visualizer`), si apre il log generato.

ASE Studio è un IDE interattivo che integra tutto l'esperimento: permette di editare il progetto, compilare, simulare e ispezionare visivamente la pipeline.

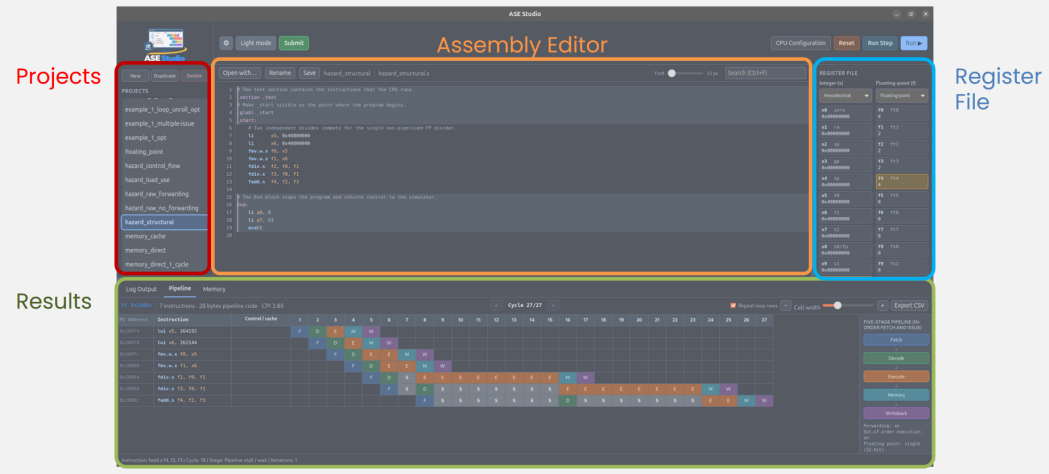


Figura 4.3: Interfaccia di ASE Studio: Editor del codice, pipeline a griglia e stato dei registri.

Nella vista della pipeline, ogni riga rappresenta un'istruzione e ogni colonna un ciclo di clock, evidenziando come l'istruzione attraversa gli stadi (F, D, E, M, W) ed eventuali stalli (S). Sulla destra è visibile lo stato del *Register File* (registri interi `x` e floating-point `f`).

Poiché il visualizzatore legge un log di una simulazione *già terminata*, **non è possibile modificare i dati in memoria o nei registri al volo** durante l'osservazione.

4.5 Stesura di un Programma: Pratiche Fondamentali

Quando si scrive un programma da valutare nel simulatore gem5, bisogna tenere a mente che stiamo misurando le prestazioni architetturali (latenze della pipeline, data hazard, etc.).

4.5.1 Il problema del Cache Miss iniziale

Nella realtà, il primo accesso a un dato in memoria richiede un trasferimento dalla memoria principale (RAM) alla memoria Cache, operazione che impiega decine o centinaia di cicli di clock. Se misurassimo il programma fin dalla prima istruzione, le nostre statistiche prestazionali sarebbero dominate e "sporcate" da questi ritardi iniziali di accesso in memoria, impedendoci di valutare l'efficienza della nostra struttura della pipeline.

Per ovviare a questo problema, si utilizza una tecnica di **pre-riscaldamento (warming) della cache**. Le linee di cache nel simulatore sono state configurate per essere molto grandi. Basterà quindi fare una *lettura a vuoto* del primo elemento di un vettore affinché l'intero blocco dati venga caricato in cache.

4.5.2 L'etichetta `_start` e la struttura del codice

Essendo in configurazione *bare metal* senza sistema operativo, il punto di ingresso **deve** chiamarsi `_start` (non `main` né `START`). Sotto questa etichetta scriveremo il "prologo" per precaricare i dati, seguito da dei `nop` per attendere che i dati arrivino effettivamente in cache (circa 4-6 cicli di clock). Solo a quel punto inseriremo l'etichetta del `main` vero e proprio, da dove inizieremo a misurare le prestazioni.

Esempio 4.2 (Somma di due Vettori con Cache Warming)**.** Di seguito un esempio per sommare due vettori (`V1` e `V2`) e salvare il risultato in un terzo vettore (`result`).

```
.section .data
```

```
V1: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
V2: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
result: .space 40   # Alloca 40 byte (10 word) senza inizializzarli

.section .text
.globl _start

_start:
# --- PROLOGO: RISCALDAMENTO CACHE ---
la x1, V1
la x2, V2
la x3, result
lw x0, 0(x1)  # Legge il primo el. di V1 (carica in cache)
lw x0, 0(x2)  # Legge il primo el. di V2 (carica in cache)
# (Il caricamento usa registri temporanei o x0, il dato non ci serve ora)
nop
nop
nop

main:
# --- INIZIO PROGRAMMA REALE ---
la x1, V1
la x2, V2
la x3, result
addi x4, x0, 10  # Contatore ciclo (10 iterazioni)

cycle:
lw x5, 0(x1)     # Legge da V1
lw x6, 0(x2)     # Legge da V2
add x7, x5, x6   # Somma
sw x7, 0(x3)     # Salva in result

# Aggiornamento indici (indirizzi byte-addressable -> +4)
addi x1, x1, 4
addi x2, x2, 4
addi x3, x3, 4
addi x4, x4, -1  # Decrementa contatore
bnez x4, cycle   # Salta a cycle se x4 != 0

END_of_PROGRAM:
# --- EPILOGO: SYSTEM CALL EXIT ---
li a0, 0         # Valore di ritorno 0
li a7, 93        # Codice syscall per exit()
ecall
```

Osservazione 4.3 (Osservazioni sul codice). Si noti che l'aggiornamento degli indici del vettore avviene sommando 4 (e non 1 come avverrebbe in linguaggio C). Questo perché la memoria è indirizzabile al byte (*byte-addressable*) e ogni numero intero (`.word`) in RISC-V occupa 32 bit, ovvero 4 byte. Per concludere correttamente la simulazione gem5 senza far crashare il sistema, il programma deve terminare con una System Call di `exit` (passando 93 al registro `a7` ed eseguendo `ecall`).

Capitolo 5

Pipelining

5.1 Introduzione e Concetti Base

Il **pipelining** è una tecnica di implementazione in cui l'esecuzione di più istruzioni viene sovrapposta nel tempo. In una pipeline, diverse unità hardware (chiamate *stadi della pipeline* o *segmenti*) completano parti diverse di istruzioni distinte in parallelo.

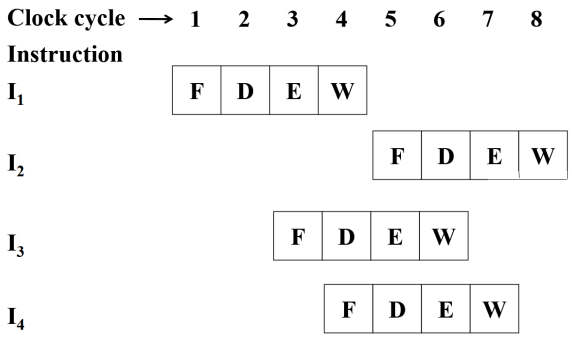


Figura 5.1: Sovrapposizione dell'esecuzione delle istruzioni nei vari cicli di clock.

5.1.1 Definizioni Fondamentali

- **Throughput**: In un processore con pipeline, il throughput è il numero di istruzioni che completano il loro percorso (ed escono dalla pipeline) nell'unità di tempo. L'obiettivo del pipelining è proprio aumentare il throughput complessivo, non velocizzare la singola istruzione.
- **Sincronizzazione degli stadi**: Tutti gli stadi della pipeline sono sincronizzati. Questo significa che procedono tutti insieme all'esecuzione del task successivo.
- **Machine Cycle (Ciclo Macchina)**: È il tempo necessario per eseguire un singolo passo (uno stadio) e normalmente corrisponde a un ciclo di clock.
- **CPI (Clock Cycles per Instruction)**: È il numero medio di colpi di clock richiesti per completare un'istruzione. L'obiettivo ideale del pipelining è portare il CPI a 1, ma come vedremo questo limite è teorico a causa dei *hazard*.

Osservazione 5.1 (La durata del ciclo di clock e lo stadio più lento). Come spiegato a lezione, la durata del machine cycle (e quindi la massima frequenza di clock del processore) è determinata **esclusivamente dallo stadio più lento**. Se abbiamo una pipeline in cui tutti gli stadi

impiegano 10 ns ma uno stadio (ad esempio l'accesso a una memoria lenta) richiede 25 ns, il clock di sistema dovrà necessariamente essere impostato a 25 ns. Questo perché tutti gli stadi devono avanzare in modo sincrono e lo stadio più lento fa da "collo di bottiglia" rallentando l'intero sistema.

5.1.2 Pipeline Ideale

In una pipeline ideale, tutti gli stadi sono *perfettamente bilanciati* (cioè richiedono esattamente lo stesso tempo per completare la loro operazione). In queste condizioni ideali, il throughput è dato dalla formula:

$$\text{Throughput}_{\text{pipelined}} = \text{Throughput}_{\text{unpipelined}} \times n$$

dove n rappresenta il numero di stadi della pipeline.

5.2 Il Datapath Unpipelined (5 Stadi)

Per comprendere i vantaggi della pipeline, partiamo dall'architettura non pipeline. L'esecuzione di ogni istruzione è divisa in un massimo di 5 cicli di clock:

1. **IF (Instruction Fetch)**: Prelievo dell'istruzione dalla memoria. Viene calcolato anche il Program Counter successivo:

$$IR \leftarrow \text{Mem}[PC]$$
$$NPC \leftarrow PC + 4$$

2. **ID (Instruction Decode / Register Fetch)**: Decodifica dell'istruzione e lettura degli operandi dal banco dei registri.

3. **EX (Execution / Effective Address)**: L'ALU esegue l'operazione o calcola l'indirizzo di memoria effettivo.

4. **MEM (Memory Access / Branch Completion)**: Accesso alla memoria dati o completamento del salto (branch).

5. **WB (Write-Back)**: Scrittura del risultato nel banco dei registri.

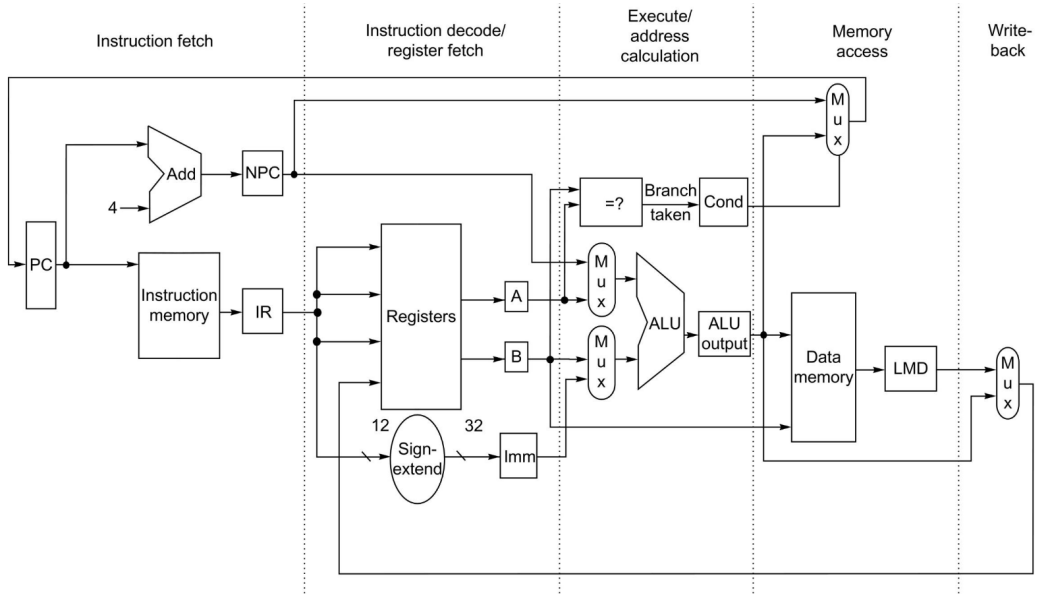


Figura 5.2: Datapath dell'architettura unpipelined, organizzato in 5 fasi logiche.

Esempio 5.1 (Valutazione delle prestazioni - Unpipelined vs Pipelined)**.** Consideriamo un processore non pipeline con un ciclo di clock di 1 ns. Supponiamo che:

- Operazioni ALU e Branch impieghino 4 cicli (frequenza: $40\% + 20\% = 60\%$).
- Operazioni di Memoria impieghino 5 cicli (frequenza: 40%).

Il tempo di esecuzione medio di un'istruzione si calcola come:

$$\text{Tempo Medio} = \text{Clock Period} \times \text{CPI}_{\text{medio}} = 1\text{ ns} \times ((0.6) \times 4 + (0.4) \times 5) = 1\text{ ns} \times 4.4 = 4.4\text{ ns}$$

Passando a un'architettura pipelined, introduciamo un overhead (ad es. ritardi dei registri). Se questo rallenta il clock del 20%, il nuovo clock sarà 1.2 ns. Nel caso ideale (un'istruzione completata per ogni ciclo), il tempo medio di esecuzione sarà proprio 1.2 ns. Lo speedup ottenuto sarà:

$$\text{Speedup} = \frac{4.4\text{ ns}}{1.2\text{ ns}} \approx 3.7\text{ volte}$$

5.3 Transizione all'Architettura Pipelined

Per far sì che l'architettura supporti la pipeline, in modo che ogni risorsa lavori su istruzioni diverse nello stesso colpo di clock, sono necessarie alcune modifiche architetturali:

- **Memorie separate**: Bisogna utilizzare memorie (o cache) separate per Istruzioni e Dati, altrimenti si verificherebbero conflitti strutturali quando uno stadio IF tenta di leggere un'istruzione nello stesso momento in cui uno stadio MEM legge o scrive un dato.
- **Gestione del Register File**: Il banco dei registri è usato da due stadi contemporaneamente: in scrittura (dallo stadio WB) e in lettura (dallo stadio ID). Viene gestito scrivendo nella prima metà del ciclo di clock (first half) e leggendo nella seconda metà (second half).

- **Pipeline Registers (Registri di interstadio)**: È essenziale inserire dei registri di stato tra uno stadio e l'altro (`IF/ID`, `ID/EX`, `EX/MEM`, `MEM/WB`) per memorizzare i dati intermedi e propagarli lungo l'esecuzione dell'istruzione nei colpi di clock successivi.

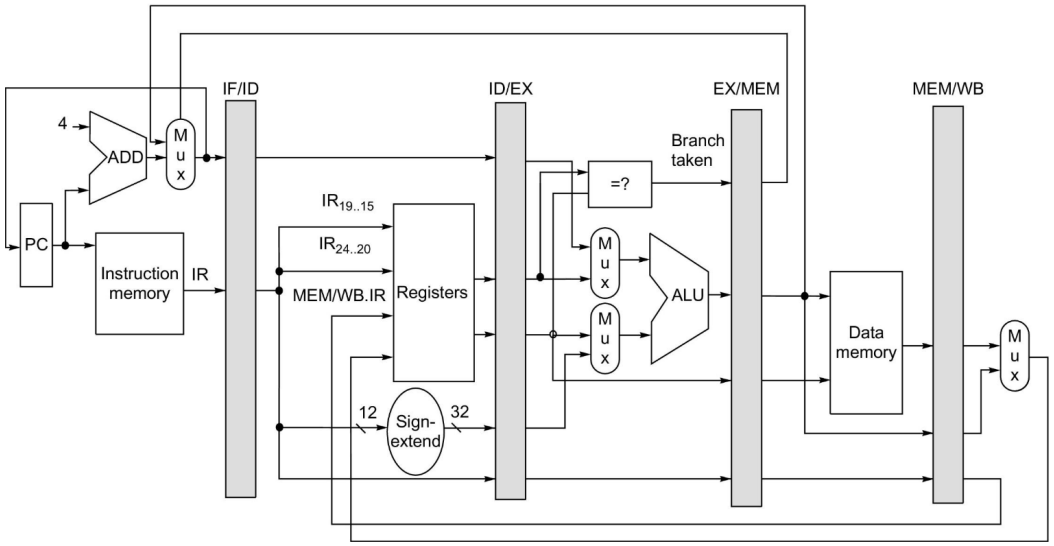


Figura 5.3: Datapath pipelined con l'introduzione dei registri di separazione tra gli stadi.

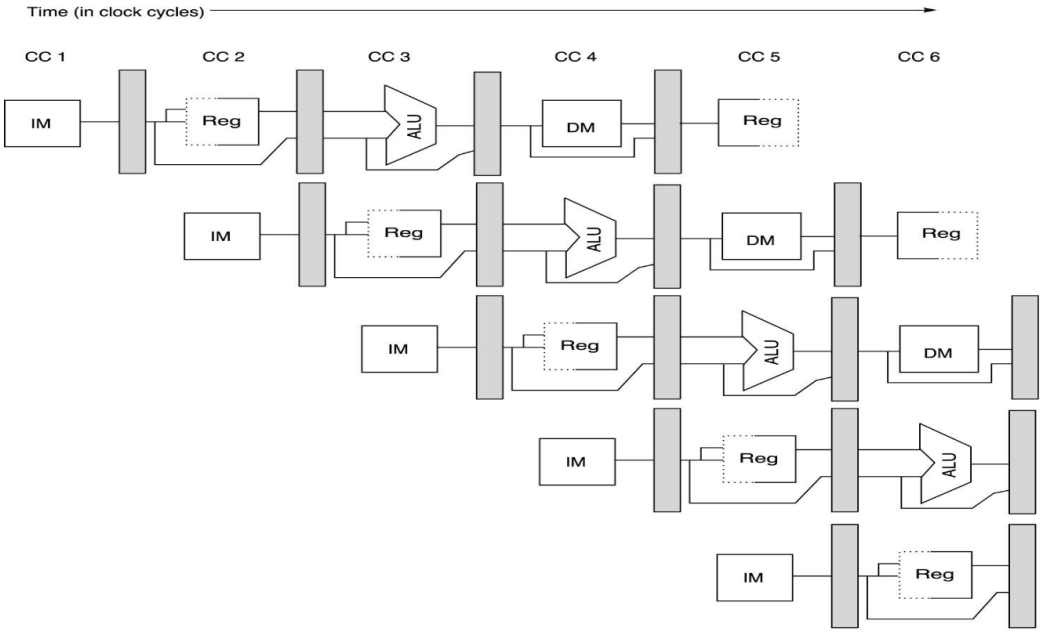


Figura 5.4: Evoluzione temporale dell'esecuzione pipelined (grafico a cascata).

5.4 Pipeline Hazards (Conflitti)

Gli **Hazards** sono situazioni anomale che impediscono a un'istruzione di essere eseguita nel suo ciclo di clock designato. Non risolti, genererebbero comportamenti non deterministici o risultati errati.

5.4.1 Data Hazards (Conflitti sui Dati)

La sovrapposizione dell'esecuzione cambia l'ordine con cui si legge e si scrive negli operandi. Se un'istruzione necessita del risultato prodotto da un'istruzione immediatamente precedente che non ha ancora raggiunto la fase di Write-Back (WB), si genera un Data Hazard.

Esempio 5.2. Consideriamo il seguente blocco di codice:

```
ADD R1, R2, R3
SUB R4, R1, R5
AND R6, R1, R7
```

L'istruzione `ADD` scriverà il valore definitivo nel registro `R1` solo nello stadio `WB` (al 5° colpo di clock). L'istruzione `SUB`, però, entra nella fase `ID` (lettura dei registri) già al 3° colpo di clock, andando a prelevare un valore di `R1` obsoleto ed errato.

Soluzioni ai Data Hazards

1. **Data Forwarding (Cortocircuitazione)**: Per alcune dipendenze, è possibile prelevare il dato direttamente all'uscita dell'ALU (nello stadio EX o MEM) e iniettarlo direttamente all'ingresso dell'ALU per l'istruzione successiva, anticipando l'attesa del ciclo WB.

2. **Stalli (Pipeline Stall o Bubble)**: Come sottolineato dal professore, in certi casi (es. operazione di `Load` da memoria dove il dato arriva fisicamente alla fine dello stadio MEM), il forwarding non è fisicamente possibile perché si richiederebbe di "mandare il dato indietro nel tempo" rispetto ai colpi di clock (vedi Fig. 5.5). In questo caso interviene la circuiteria di interlock che inserisce uno **stallo** (chiamato *bubble* o bolla). La bolla transita per tutti gli stadi successivi, rallentando l'esecuzione dell'istruzione dipendente fintanto che il dato non è disponibile per essere forwardato correttamente.

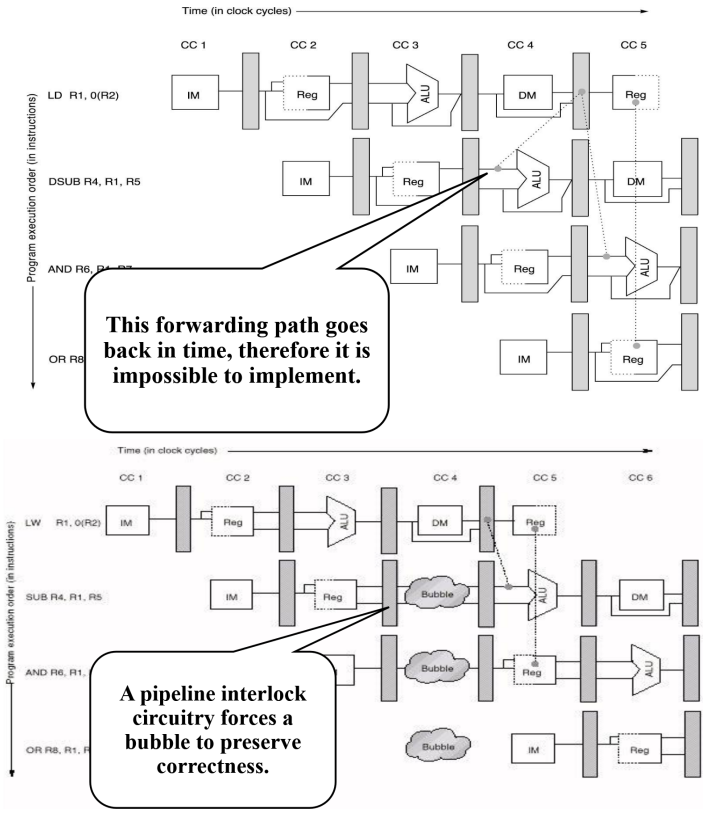


Figura 5.5: Risoluzione di un Load-use Data Hazard inserendo una Bubble (bolla) nella pipeline per ritardare l'istruzione dipendente.

5.4.2 Control Hazards (Conflitti di Controllo)

I conflitti di controllo sono dovuti alle istruzioni di salto (*branches*, condizionati o incondizionati). Il problema nasce perché il salto può modificare il Program Counter (`PC`), ma questa decisione (ovvero capire *se* si salta e *dove* si salta) viene presa quando l'istruzione successiva è già entrata nella pipeline. Nella versione base della pipeline, il target del PC viene scritto alla fine dello stadio `EX` (ovvero 2 colpi di clock dopo la fase IF in cui il branch è stato caricato), obbligando il processore ad annullare ben due istruzioni successive già processate se il salto è *preso* (taken).

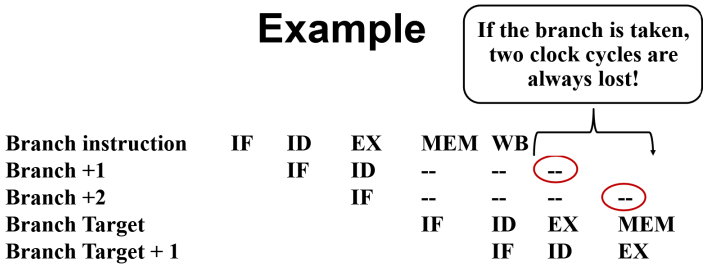


Figura 5.6: Esempio di Control Hazard: se il branch è preso, le due istruzioni successive vengono annullate, causando una perdita di cicli di clock.

Soluzioni ai Control Hazards

- **Anticipazione (Basic Solution)**: La circuiteria del branch viene spostata nello stadio **ID**. Si anticipa quindi sia la decisione se saltare, sia il calcolo del target `PC`. Lo stallo viene ridotto, ma si inserisce hardware aggiuntivo nel datapath in fase ID.
- **Delayed Branch (Salto ritardato)**: Il compilatore inserisce nel *delay slot* (lo spazio temporale subito dopo il branch) istruzioni indipendenti e sicure da eseguire, indipendentemente dall'esito del salto.

  Osservazione 5.2 (Evoluzione tecnologica). Oggi questa tecnica è diventata obsoleta. Con l'avvento di processori con *deep pipeline* (pipeline molto profonde, anche da 15-20 stadi), la lunghezza dei delay slot aumenta vistosamente, rendendo quasi impossibile per il compilatore trovare abbastanza istruzioni utili per riempirli. Pertanto, gran parte delle moderne architetture RISC non li supporta più, preferendo meccanismi complessi di *Branch Prediction* (previsione di salto).

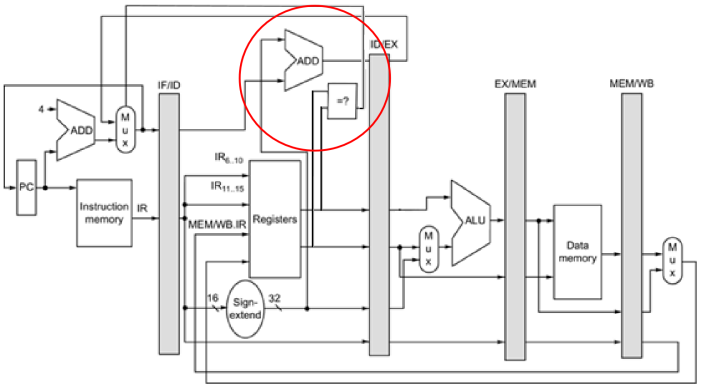


Figura 5.7: Spostamento della logica di salto (calcolo dell'indirizzo e confronto) nello stadio ID per mitigare la penalty da control hazard.

Capitolo 6

Operazioni Multiciclo (Multicycle Operations)

6.1 Introduzione alle Operazioni Floating-Point

Nei capitoli precedenti abbiamo analizzato la pipeline per le istruzioni intere e abbiamo visto come risolvere problemi relativi ai conflitti (hazards) tramite tecniche come il forwarding o modifiche architetturali (es. spostamento del calcolo del salto).

Quando però introduciamo istruzioni per la gestione dei numeri reali (rappresentati in virgola mobile tramite lo standard IEEE 754), la situazione si complica notevolmente. Le unità *Floating-Point* (FP) devono eseguire operazioni molto più complesse rispetto a quelle intere. Banalmente, una moltiplicazione in virgola mobile richiede non solo la moltiplicazione delle mantisse, ma anche la somma degli esponenti e successive normalizzazioni.

Per gestire questa complessità e forzare queste unità a completare il loro lavoro in un singolo ciclo di clock, il progettista hardware si troverebbe di fronte a due scelte estreme:

- utilizzare un clock estremamente lento (penalizzando le istruzioni semplici);
- creare unità hardware gigantesche e incredibilmente complesse.

Poiché entrambe le soluzioni sono inefficienti (un'area di silicio troppo grande o prestazioni generali degradate), l'alternativa comunemente adottata è permettere alle operazioni FP di richiedere **più di un ciclo di clock** per completare l'esecuzione. In questa architettura, lo stadio di *Execute* (EX) non è più un blocco monolitico, ma è composto da diverse unità funzionali e viene ripetuto o iterato tante volte quanto richiesto dalla specifica istruzione.

6.2 Estensione della Pipeline per il Floating-Point

La pipeline classica per gli interi è composta dai classici 5 stadi: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory access) e WB (Write Back).

Per supportare le operazioni multiciclo, la fase EX viene espansa in parallelo con moduli dedicati.

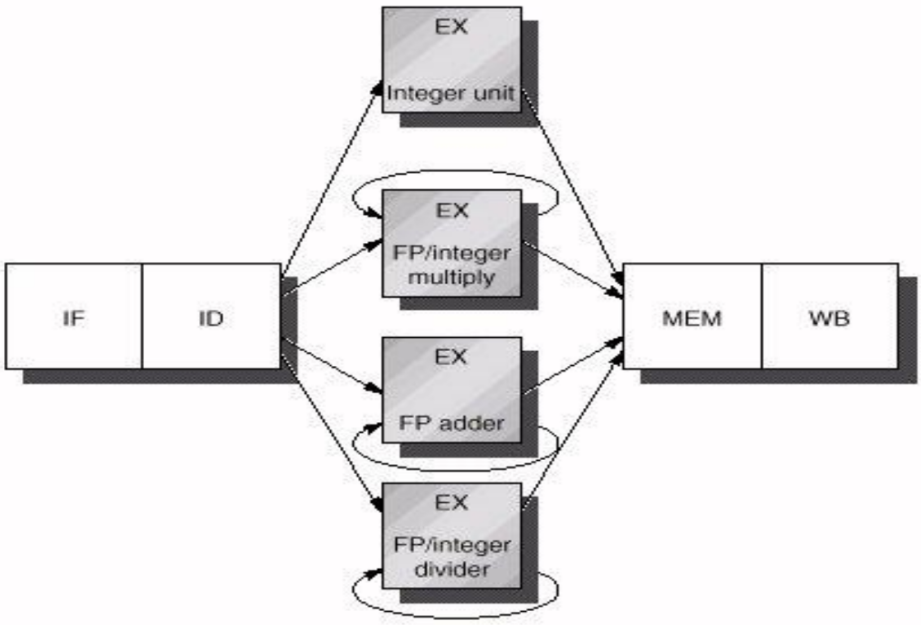


Figura 6.1: Estensione della pipeline per supportare le operazioni Floating-Point.

Come si nota dallo schema, dopo la fase di decodifica (ID), l'istruzione viene indirizzata all'unità funzionale appropriata. Alcune di queste unità (come l'addizionatore o il moltiplicatore FP) possiedono dei "cicli" (loop) su se stesse, a indicare che il dato permane all'interno dell'unità per più cicli di clock prima di passare alla fase MEM.

6.3 Latenza e Intervallo di Inizializzazione

Per definire il comportamento di queste nuove unità funzionali che richiedono più di un ciclo, dobbiamo introdurre due concetti fondamentali.

Definizione 6.1 (Latenza)**.** È il numero di cicli di clock che devono intercorrere tra un'istruzione che produce un risultato e un'istruzione successiva che utilizza quello stesso risultato.

Osservazione 6.1. Nella pipeline intera classica, grazie alla rete di *forwarding* (che porta il dato dalla fine della fase EX direttamente all'inizio della nuova fase EX), abbiamo di fatto "azzerato" la latenza. Nel caso del Floating-Point, a causa dell'elaborazione più lunga, il dato non sarà subito disponibile e la latenza tornerà a essere un fattore limitante.

Definizione 6.2 (Intervallo di Inizializzazione (Initiation Interval))**.** È il numero di cicli di clock che devono trascorrere tra l'avvio (issue) di due operazioni dello stesso tipo indirizzate alla medesima unità funzionale.

Per comprendere meglio, osserviamo un tipico esempio dei tempi richiesti dalle varie unità.

Functional Unit	Latency	Initiation Interval
Integer ALU	0	1
Data Memory	1	1
FP add	3	1
FP/integer multiply	6	1
FP/integer divide	24	25

Tabella 6.1: Esempio di Latenze e Intervalli di Inizializzazione.

6.3.1 Unità Pipelined e Non Pipelined

Guardando la Tabella 6.1, possiamo dedurre una caratteristica architetturale cruciale:

- **Unità Pipelined (es. FP add, FP multiply):** Hanno un intervallo di inizializzazione pari a 1. Questo significa che, sebbene l'operazione richieda più cicli per concludersi (es. 4 cicli per la somma, divisi in stadi A1, A2, A3, A4), la struttura interna è a sua volta segmentata in pipeline. Possiamo quindi inserire una nuova somma a ogni ciclo di clock.
- **Unità Non Pipelined (es. FP divide):** L'intervallo di inizializzazione è maggiore o uguale alla latenza (es. 25). Questo significa che l'unità di divisione non è segmentata in stadi indipendenti: una volta avviata una divisione, l'unità è "occupata" per l'intera durata del calcolo. Bisogna attendere che finisca prima di potergliene inviare un'altra.

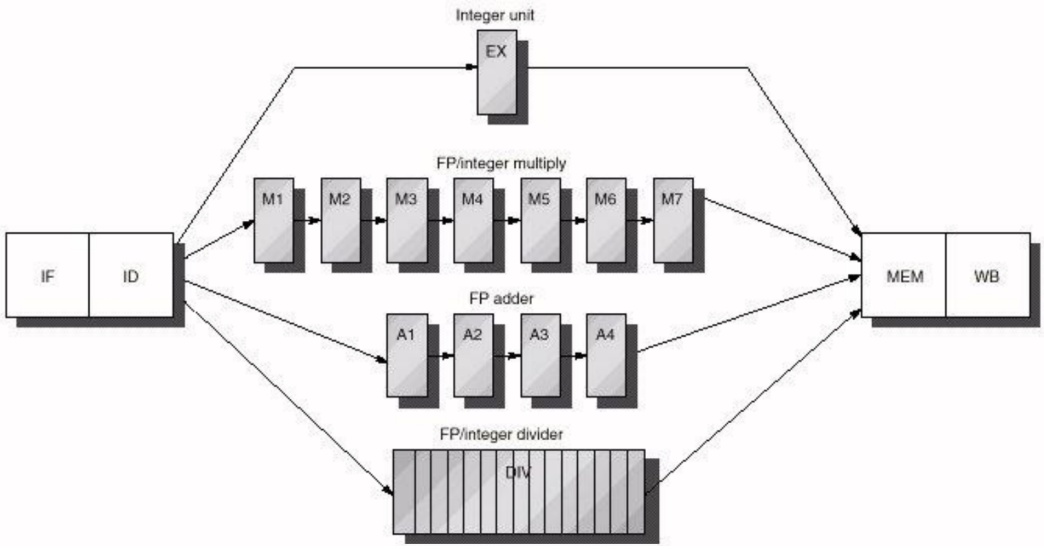


Figura 6.2: Dettaglio degli stadi interni delle unità funzionali (Multiply e Add sono pipelined, Divide non lo è).

6.4 Conflitti (Hazards) nelle Pipeline Multiciclo

A causa della struttura eterogenea dello stadio EX, la frequenza dei conflitti (hazards) aumenta significativamente. Il processore cercherà di eseguire (eseguire il *fetch*) un'istruzione a ogni colpo di clock, ma dovrà ritardare la pipeline quando le condizioni non lo permettono.

6.4.1 Conflitti Strutturali (Structural Hazards)

I conflitti strutturali si verificano in due situazioni principali introdotte dalle operazioni multiciclo:

1. **Condivisione di unità non pipelined:** Se due istruzioni indipendenti di divisione (es. la prima divide `F2/F3` e la seconda `F5/F6`) arrivano ravvicinate, la seconda dovrà stallare nello stadio di decodifica (ID) perché l'unità DIV è tenuta occupata dalla prima istruzione.

2. **Conflitti sul Register File (Scritture simultanee):** Poiché le istruzioni hanno tempi di esecuzione diversi, è possibile che istruzioni partite in momenti diversi terminino esattamente nello stesso ciclo di clock. Se la pipeline possiede un'unica porta di scrittura (Write port) verso il Register File, non sarà in grado di gestire più di un'operazione di *Write Back* (WB) nello stesso ciclo. Il processore dovrà inserire degli stalli per sequenzializzare le scritture.

	Clock cycle number										
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
`fmul.d f0,f4,f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
`fadd.d f2,f4,f6`				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
`fld f2,0(x2)`							IF	ID	EX	MEM	WB

Figura 6.3: Esempio di conflitti strutturali dovuti a scritture simultanee sul Register File.

6.4.2 Conflitti Dati e Forwarding

Quando due istruzioni Floating-Point sono dipendenti tra loro (la seconda utilizza il risultato della prima), il meccanismo di forwarding deve essere riadattato.

Esempio 6.1. Sintassi delle operazioni in virgola mobile (es. architettura RISC-V/MIPS): `fmul.d` indica una moltiplicazione per numeri double (64 bit), mentre `fmul.s` indica single precision (32 bit).

Supponiamo di avere due moltiplicazioni dipendenti:

1. `fmul.d F1, F2, F3` (Scrive in F1)
2. `fmul.d F4, F1, F5` (Legge F1)

La seconda istruzione non può procedere. Verrà bloccata nella fase di decodifica (ID) finché la prima istruzione non raggiunge l'ultimo stadio della sua esecuzione (es. M7). A quel punto, il risultato potrà subire un *forwarding* verso il primo stadio di esecuzione (M1) della seconda istruzione. In questo nuovo scenario, le reti di forwarding diventano molto più complesse, poiché vi sono "molti più sorgenti di dato e molte più destinazioni di dato" rispetto all'architettura per interi.

Osservazione 6.2. Se le istruzioni *non* sono dipendenti (es. due moltiplicazioni su registri diversi), la seconda istruzione segue normalmente la prima nella pipeline. Entrerà nello stadio M1 non appena la prima passa allo stadio M2, garantendo così una perfetta parallelizzazione del calcolo.

6.5 Moduli e Fasi Operative del Floating-Point

L'unità FP, data la sua complessità, può essere logicamente scomposta in 8 stadi funzionali base, che vengono ricombinati a seconda dell'operazione.

Stage	Functional Unit	Description
A	Adder	Mantissa ADD stage
D	Divider	Divide
E	Multiplier	Exception test
M	Multiplier	Multiplier I
N	Multiplier	Multiplier II
R	Adder	Rounding
S	Adder	Operand shift
U	(N/A)	Unpack numbers

Tabella 6.2: Componenti di base dell'unità Floating-Point.

Questi blocchi compongono le varie istruzioni. Ad esempio, una somma FP (Add, subtract) passerà per gli stadi di Unpack, Shift, Add e Rounding.

Osservazione 6.3 (Ottimizzazione della Latenza). La latenza effettiva per l'uso del dato può essere ridotta di un ciclo di clock (latenza - 1) se l'istruzione di destinazione (quella che attende il dato) non è un'operazione aritmetica, bensì un'istruzione di memorizzazione (*Store instruction*, es. `fsd`). Questo perché la Store necessita del dato in ingresso solo allo stadio MEM, non in EX.

6.6 Prestazioni (Performance)

Introdurre istruzioni che richiedono così tanti cicli di clock ha un impatto visibile sul *Cycles Per Instruction* (CPI) medio della macchina.

Dall'analisi delle prestazioni si evince che il comportamento del processore cambia drasticamente in base alla natura del programma eseguito:

- **Programmi Interi:** La causa predominante di decadimento delle prestazioni (aumento del CPI) sono i ritardi legati ai salti (*Branch delays*).
- **Programmi Floating-Point:** Le cause principali di stallo diventano i conflitti sui dati FP (*FP result stalls*, dovuti all'alta latenza delle operazioni) e, in misura minore, i conflitti strutturali.

A seconda del programma, il CPI totale può variare ampiamente in un range compreso tra 1.2 e 2.8.

Capitolo 7

Exceptions and Interrupts in Pipelined Processors

Le eccezioni e le interruzioni sono eventi che modificano il normale ordine di esecuzione di un programma. Nelle architetture con pipeline, la gestione di questi eventi eccezionali (eccezioni, interruzioni o fault) risulta particolarmente complessa poiché ci sono più istruzioni in esecuzione contemporaneamente in diversi stadi della pipeline.

7.1 Distinzione tra Eccezione e Interruzione

I termini "eccezione" e "interruzione" vengono spesso confusi o usati in modo intercambiabile (ad esempio, nell'ecosistema ARM si parla quasi esclusivamente di eccezioni per entrambi i casi). Tuttavia, a livello teorico, esiste una distinzione precisa basata sull'origine dell'evento:

- **Eccezione (Exception):** Si riferisce generalmente a un evento interno alla CPU, generato dall'esecuzione stessa di un'istruzione. Esempi classici sono:
  - Overflow in operazioni aritmetiche (es. Floating Point overflow o divisione per zero).
  - Fault della MMU (es. Page fault, accesso non allineato in memoria).
  - Trap o Software Interrupt (SWI), ovvero istruzioni non definite.
- **Interruzione (Interrupt):** Si riferisce solitamente a un evento esterno di I/O, asincrono rispetto al flusso del programma. Esempi tipici sono:
  - Richiesta da un dispositivo di I/O (es. tocco sullo schermo, ricezione di dati sulla porta seriale).
  - Segnale di Reset o calo di alimentazione della batteria.

Osservazione 7.1 (Sistemi Bare-metal vs. Sistemi Operativi). In un sistema operativo completo, le eccezioni sono spesso gestite dal kernel. Lavorando invece "bare-metal" (senza sistema operativo), come si fa abitualmente con microcontrollori RISC-V o ARM nei corsi base, il programmatore deve scrivere esplicitamente le *Interrupt Service Routine* (ISR) per decidere come il processore deve reagire a ogni singolo evento (es. assegnare un valore massimo in caso di divisione per zero o ripetere l'operazione).

7.2 Classificazione delle Eccezioni

Gli eventi eccezionali possono essere classificati secondo cinque categorie principali:

7.2.1 Sincrone vs. Asincrone

- **Sincrone:** L'evento si verifica sempre nello stesso punto del codice se il programma viene rieseguito con gli stessi dati. Sono eccezioni generate dalle istruzioni stesse.
- **Asincrone:** Eventi esterni (es. timer, periferiche I/O) che non dipendono dall'istruzione in esecuzione e possono avvenire in qualsiasi momento.

Esempio 7.1 (Emulazione floating-point tramite eccezioni sincrone)**.** Se un processore non possiede unità hardware per i numeri reali (Floating Point Unit), un'istruzione come `fmul.s` genererà un'eccezione di "istruzione non definita". Il programmatore può sfruttare questa eccezione sincrona per far saltare il processore a una routine software che emula la moltiplicazione reale usando le unità intere, per poi restituire il risultato e riprendere l'esecuzione.

7.2.2 User Requested vs. Coerced (Richieste vs. Forzate)

- **User Requested:** L'utente o il programmatore richiede esplicitamente l'eccezione, in modo simile a una chiamata a procedura (es. *Breakpoint* per il debugging passo-passo).
- **Coerced (Forzate):** Sono al di fuori del controllo del programma utente (es. page fault, hardware malfunction).

7.2.3 Maskable vs. Non-maskable (Mascherabili vs. Non mascherabili)

- **Mascherabili:** L'hardware può essere configurato per ignorare (mascherare) temporaneamente la richiesta, dandole una priorità inferiore.
- **Non-maskable:** Eventi critici che il processore deve servire immediatamente, indipendentemente da cosa sta facendo. Un classico esempio è il segnale di calo di alimentazione (power failure).

7.2.4 Within vs. Between Instructions (All'interno vs. Tra le istruzioni)

Le eccezioni possono verificarsi *all'interno* dell'esecuzione di una singola istruzione (impedendone il completamento) o *tra* un'istruzione e la successiva. In architetture pipeline, la distinzione sfuma perché più istruzioni si sovrappongono.

7.2.5 Resume vs. Terminate (Ripresa vs. Terminazione)

Dopo aver gestito l'eccezione, il programma può terminare l'esecuzione oppure riprendere.

Osservazione 7.2 (Restartable Machines e Watchdog)*.* Oggi quasi tutti i processori sono macchine "restartable": sono in grado di gestire un'eccezione, salvare lo stato e ripartire. Tuttavia, in sistemi *safety-critical* (es. centraline automobilistiche), ripartire a tutti i costi non è sempre l'opzione migliore. Spesso si utilizza un **Watchdog Timer**, che genera un'interruzione a intervalli regolari: se il sistema è in salute, il watchdog viene "resettato" dal software; se il software si blocca, il watchdog fa scattare un reset completo del sistema.

7.3 Meccanismo di Gestione nella Pipeline

Quando si verifica un'eccezione, per poter fermare e poi riavviare (Restart) l'esecuzione, la pipeline deve eseguire i seguenti passi:

1. Forzare un'istruzione di *trap* nella pipeline al successivo stadio di Fetch (IF).
2. Finché la trap non viene presa, **spegnere tutte le scritture** (registri o memoria) per l'istruzione che ha causato l'eccezione e per tutte le istruzioni successive presenti nella pipeline (per evitare che i dati o gli indici si corrompano, come nel caso di autoincremento).
3. Il controllo passa alla procedura di gestione (*exception-handling procedure*), che salva immediatamente il Program Counter (PC) dell'istruzione a cui bisognerà fare ritorno.

7.3.1 Protocolli di Interruzione a confronto

Diversi processori gestiscono il salto all'Interrupt Service Routine (ISR) in modo diverso.

Architettura 80x86 (Hardware Vectoring)

1. Il dispositivo esterno invia la richiesta. La CPU rileva l'interruzione.
2. La CPU legge il numero di identificazione N (Interrupt Type) dal bus.
3. Salva **automaticamente** nello stack il Processor Status Word (PSW), il Code Segment (CS) e l'Instruction Pointer (IP). Tutti gli altri registri usati dall'utente devono essere salvati manualmente.
4. Azzera i flag delle interruzioni per disabilitare altre interruzioni esterne.
5. Usa N come indice per cercare nella **Interrupt Vector Table (IVT)** l'indirizzo a cui saltare.

Architettura ARM

Simile all'80x86, usa una Interrupt Vector Table ed N, ma è più "gentile" con il programmatore: salva automaticamente nello stack un *frame* di **8 registri** (secondo le standard ABI), tra cui il Program Counter e il Processor Status Register. Aggiorna i flag e salta all'indirizzo dell'ISR ricavato dalla tabella.

Architettura RISC-V

Il RISC-V, nella sua implementazione base, non possiede una Interrupt Vector Table hardware automatica con indice N.

1. Salva il PC corrente e la **causa dell'eccezione** in due registri speciali dedicati.
2. Salta a un singolo indirizzo predefinito (*exception handler address*).
3. È compito dell'utente/programmatore scrivere un blocco di codice (`switch/case`) in questo handler che legga il registro della causa e chiami la corretta funzione software (es. divisione per zero, timer, ecc.).

7.4 Eccezioni Precise e Imprecise

Un processore possiede **eccezioni precise** se, all'arresto della pipeline per un'eccezione, si garantisce che:

- Tutte le istruzioni precedenti a quella che ha scatenato l'eccezione sono state completate.

- Nessuna delle istruzioni successive è stata completata o ha modificato in modo permanente lo stato della macchina (possono essere riavviate da zero).

Garantire eccezioni precise è complesso quando si usano istruzioni multi-ciclo, specialmente con esecuzione fuori ordine (*out-of-order completion*).

Esempio 7.2 (Completamento fuori ordine e Stato Impreciso)**.** Consideriamo il seguente frammento di codice con istruzioni Floating Point, che tipicamente impiegano molti più cicli delle istruzioni intere (es. 24 cicli per la divisione):

```
fdiv.d f0, f2, f4   // Richiede molti cicli
fadd.d f10, f10, f8 // Termina prima della fdiv
fsub.d f12, f12, f14
```

Non essendoci dipendenze sui dati, `fadd.d` e `fsub.d` termineranno **prima** di `fdiv.d`. Se durante `fsub.d` si verifica un'eccezione, ci troviamo in uno stato impreciso: la `fdiv.d` (precedente) non è completa, ma la `fadd.d` (successiva) ha già sovrascritto il registro `f10`.

Soluzioni possibili per le eccezioni imprecise:

- Accettare le eccezioni imprecise (sconsigliato).
- Fornire una modalità "lenta ma precisa" e una "veloce ma imprecisa" (eseguendo una sola operazione FP alla volta).
- Analizzare in anticipo (early determine) se le unità FP provocheranno eccezioni, frenando il fetch.
- **Bufferizzazione (Commit Stage):** Inserire i risultati temporanei in un buffer e scriverli permanentemente nei registri (fase di Write-Back) solo quando tutte le istruzioni precedenti sono state completate con successo. Questa è la strategia principale nei processori moderni (superscalari).

7.5 Complicazioni nella Pipeline e Ordine delle Eccezioni

Nel RISC-V base, i vari stadi della pipeline possono generare eccezioni differenti:

Stadio della Pipeline	**Causa dell'eccezione**
IF (Instruction Fetch)	Page fault sull'istruzione, accesso non allineato, violazione protezione.
ID (Instruction Decode)	Opcode indefinito o illegale.
EX (Execution)	Eccezione aritmetica (es. overflow).
MEM (Memory)	Page fault sui dati, accesso non allineato, violazione protezione.
WB (Write Back)	Nessuna (lo stadio non genera eccezioni).

Tabella 7.1: Sorgenti di eccezione nei vari stadi della pipeline RISC-V.

A causa dell'avanzamento parallelo, possono presentarsi due problematiche critiche:

7.5.1 Eccezioni Contemporanee

Un'istruzione precedente si trova nello stadio MEM e va in fault, e un'istruzione successiva si trova nello stadio EX e va in fault contemporaneamente nello stesso ciclo di clock.

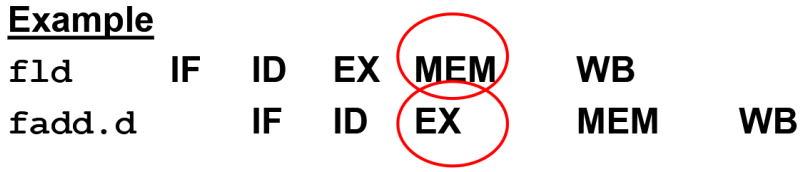


Figura 7.1: Eccezioni contemporanee in stadi diversi della pipeline.

La strategia prevede di processare sempre l'eccezione dell'istruzione "più vecchia" (quella entrata per prima, in questo caso la `fld`).

7.5.2 Eccezioni in ordine inverso

Un'istruzione successiva genera un'eccezione in uno stadio "precoce" (es. IF) prima che un'istruzione precedente generi un'eccezione in uno stadio "tardivo" (es. MEM).

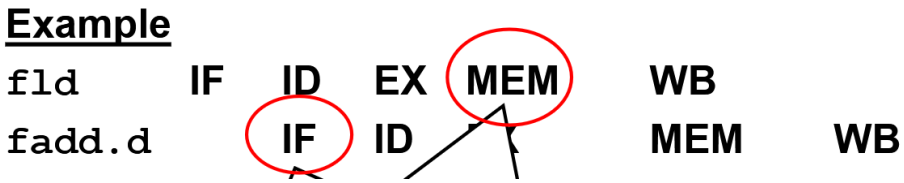


Figura 7.2: Eccezioni che si verificano in ordine opposto rispetto alle istruzioni.

Soluzione per le pipeline RISC:

- Ad ogni istruzione viene associato uno **status flag** (o vettore di flag) che viaggia con essa nella pipeline.
- Se un'istruzione causa un'eccezione, non viene interrotto subito il processore, ma viene settato il suo status flag.
- Un'istruzione con il flag settato procede, ma le viene inibito l'accesso in scrittura (niente side-effect sulla memoria o sui registri).
- Solo quando l'istruzione raggiunge l'ultimo stadio (commit/WB), il processore controlla il flag. Se è settato, solleva ufficialmente l'eccezione, garantendo l'ordine cronologico del programma originale.

7.5.3 Altre Complicazioni del Set di Istruzioni

- **Modifiche di stato pre-commit:** Istruzioni come quelle con indirizzamento ad *autoincremento* modificano lo stato (i registri indice) prima del completamento dell'istruzione. Se abortite, lasciano uno stato alterato, costringendo il processore a logiche di *roll-back* molto complesse per ripristinare il valore precedente dell'indice.
- **Condition Codes (Flag di stato):** Presenti in ARM e 80x86 (ma assenti in RISC-V). Causano data hazards impliciti, costringono al salvataggio e ripristino durante un'eccezione e complicano il riempimento dei *branch delay slot*.

- **Istruzioni complesse (CISC):** Sono difficili da pipelinare mantenendo stadi di lunghezza costante. Spesso si risolve pipelinando le *microistruzioni* che compongono l'istruzione complessa.

Capitolo 8

Instruction Level Parallelism (ILP)

8.1 Introduzione

I processori con architettura pipeline sfruttano il parallelismo esistente tra le istruzioni, noto come **Instruction-Level Parallelism (ILP)**, che permette la loro esecuzione in parallelo. L'obiettivo principale della progettazione di queste architetture è aumentare la quantità di istruzioni diverse che vengono eseguite simultaneamente nelle diverse fasi della pipeline. Più è alta la quantità di ILP che può essere trovata e sfruttata, migliori saranno le prestazioni (performance) della pipeline.

8.2 Approcci allo sfruttamento dell'ILP

Esistono fondamentalmente due approcci per sfruttare l'ILP per evitare o mitigare i problemi derivanti dalle dipendenze di dato e dagli hazard strutturali o di controllo:

- **Approccio Dinamico (Hardware-based):** È l'hardware stesso a localizzare il parallelismo in fase di esecuzione (es. architettura Tomasulo). Domina i mercati desktop e server ed è presente in processori molto spinti (es. Intel Core Series, ARM Cortex-A9, MIPS R10000/12000, RISC-V avanzati).
- **Approccio Statico (Software-based):** È definito dal compilatore prima dell'esecuzione. Trovava grande utilizzo in passato e domina ancora nel mercato dei sistemi *embedded* (es. ARM Cortex-A8) poiché non richiedono elaborazioni hardware così spinte. Il ruolo del "compilatore" in questo contesto sarà svolto da noi, operando scelte strategiche sul codice Assembly.

I due approcci possono anche essere parzialmente combinati.

8.3 Parallelismo nei Basic Block

Per parlare di parallelizzazione statica, dobbiamo partire dal concetto fondamentale di *Basic Block*. Il primo tipo di ILP si cerca tra le istruzioni che appartengono allo stesso blocco base.

Definizione 8.1 (Basic Block)**.** Un *basic block* è una sequenza compatta di istruzioni che presenta le seguenti caratteristiche:

- Non ha salti in ingresso, se non il punto di ingresso al blocco stesso.
- Non ha salti in uscita, se non il punto di uscita dal blocco.

In altre parole, nessuna istruzione entra in un punto intermedio e nessuna esce in un punto intermedio.

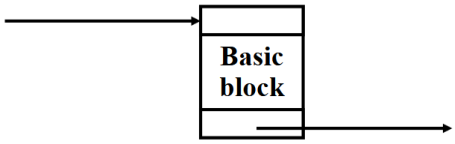


Figura 8.1: Struttura concettuale di un Basic Block.

Esempio 8.1. Consideriamo il seguente frammento di codice ad alto livello in C:

```
while (save[i] == k)
    i += 1;
```

Il compilatore trasforma questo frammento in Assembly. Il codice risultante può essere suddiviso in due Basic Block, uno contenente le operazioni di caricamento e confronto (accesso alla memoria `save[i]` e branch), e uno contenente l'incremento dell'indice e il salto incondizionato.

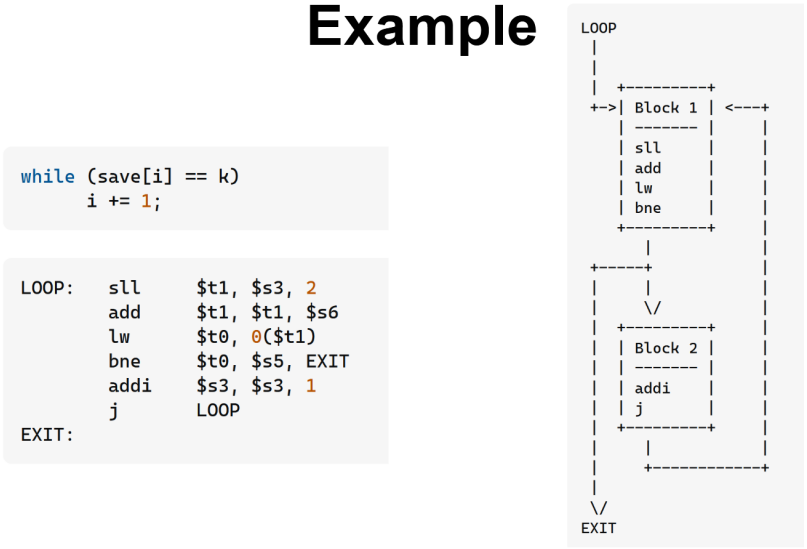


Figura 8.2: Esempione di suddivisione del codice in Basic Blocks.

Osservazione 8.1. **Il limite dei Basic Block:** Per i tipici programmi RISC-V, la dimensione di un basic block è molto piccola, solitamente tra le 4 e le 7 istruzioni. Poiché è altamente probabile che queste poche istruzioni siano dipendenti l'una dall'altra, la quantità di parallelismo sfruttabile all'interno di un singolo blocco base è normalmente piuttosto ridotta.

8.4 Tecniche di Ottimizzazione Statica (Software)

8.4.1 Rescheduling (Rischedulazione)

All'interno di un basic block, il compilatore (o il programmatore) può rischedulare, ovvero spostare in una posizione diversa, le istruzioni per ottimizzare il codice ed evitare stalli dovuti

a dipendenze di dato reali (es. dati necessari non ancora pronti a causa di latenze della memoria).

Esempio 8.2. Vogliamo eseguire le seguenti due operazioni ad alto livello:

$$a = b + c; \quad d = e - f;$$

Assumiamo che le istruzioni di `load` abbiano una latenza di un colpo di clock. Supponendo che i registri base da `x10` a `x15` siano già stati impostati, il codice Assembly non ottimizzato sarebbe:

```
lw x1, 0(x10)
lw x2, 0(x11)
add x5, x1, x2  // Dipendenza da x2 e x1
sw x5, 0(x14)
lw x3, 0(x12)
lw x4, 0(x13)
add x6, x3, x4  // Dipendenza da x4 e x3
sw x6, 0(x15)
```

Se analizziamo l'evoluzione in pipeline di questo codice non ottimizzato, notiamo una dipendenza: `x2` è destinazione di una `load` e sorgente della `add` immediatamente successiva. Essendo `x2` disponibile solo dopo la fase `MEM`, la `add` deve stallare in fase di decodifica, facendo perdere cicli di clock. Questo si ripete per la seconda operazione. Il programma richiede **14 colpi di clock**.

Per evitare questo, applichiamo il **rescheduling**: spostiamo le `load` della seconda operazione in alto, prima delle operazioni aritmetiche, riempiendo così gli stalli (visto che si tratta di calcoli indipendenti).

Codice Non Ottimizzato	**Codice Rischedulato**
`lw x1, 0(x10)`	`lw x1, 0(x10)`
`lw x2, 0(x11)`	`lw x2, 0(x11)`
`add x5, x1, x2` *(stallo)*	`lw x3, 0(x12)`
`sw x5, 0(x14)`	`add x5, x1, x2`
`lw x3, 0(x12)`	`lw x4, 0(x13)`
`lw x4, 0(x13)`	`sw x5, 0(x14)`
`add x6, x3, x4` *(stallo)*	`add x6, x3, x4`
`sw x6, 0(x15)`	`sw x6, 0(x15)`

Tabella 8.1: Confronto tra codice non ottimizzato e ottimizzato tramite rescheduling.

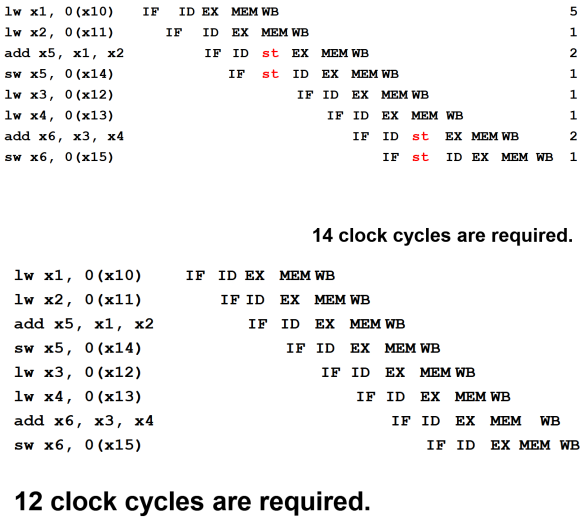


Figura 8.3: Evoluzione in pipeline del codice rischedulato, mostrando l'eliminazione degli stalli.

Con il codice rischedulato, nessun `load stall` è richiesto, e l'esecuzione si conclude in **12 colpi di clock**, risparmiando tempo prezioso.

8.4.2 Loop-Level Parallelism e Loop Unrolling

Per incrementare ulteriormente il parallelismo disponibile, non ci si ferma al singolo Basic Block (troppo piccolo), ma si considera il parallelismo tra iterazioni di un ciclo (*loop*). Due modi principali per farlo sono:

1. **SIMD (Single Instruction, Multiple Data):** Usato da processori vettoriali e GPU, dove diverse unità funzionali eseguono task simili in parallelo su dati multipli con un singolo colpo di clock.

2. **Loop Unrolling (Srotolamento dei cicli):** Tecnica (statica o dinamica) in cui il corpo del loop viene esplicitamente replicato più volte all'interno dell'iterazione.

Definizione 8.2 (Loop Unrolling)**.** Il Loop Unrolling consiste nello srotolare un ciclo replicando il suo body. Ad esempio, invece di eseguire N iterazioni saltando indietro ogni volta, si eseguono $N/4$ iterazioni contenenti ciascuna 4 repliche del body (aggiornando opportunamente gli indici e i contatori).

Esempio 8.3. Trasformazione del ciclo in C:

```
// Ciclo Originale
for (i=0; i<N; i++) {
    x[i] = x[i] + y[i];
}

// Ciclo dopo Unrolling (fattore 4)
for (i=0; i<N; i=i+4) {
    x[i] = x[i] + y[i];
    x[i+1] = x[i+1] + y[i+1];
    x[i+2] = x[i+2] + y[i+2];
    x[i+3] = x[i+3] + y[i+3];
}
```

Osservazione 8.2. **Vantaggi dell'Unrolling:**

- L'overhead relativo al controllo dell'iterazione si riduce enormemente. I salti (branch) fanno perdere colpi di clock (penalità). Se passo da 100 salti a 25 salti, sto risparmiando ben 75 colpi di clock solo in overhead di salto.
- Il corpo del ciclo diventa molto più ampio (creando un Basic Block "virtualmente" molto più grande), aumentando le possibilità per il compilatore di usare il *rescheduling* per eliminare gli stalli.

Svantaggi:

- Aumenta la dimensione del codice (Code Size), poiché il corpo viene fisicamente moltiplicato in memoria.

8.5 Dipendenze e Hazard

Le dipendenze sono proprietà intrinseche del programma. Esse determinano l'ordine in cui i risultati devono essere calcolati, pongono un limite superiore al parallelismo sfruttabile e creano la possibilità che si verifichi un *hazard* (rischio di stallo) nell'architettura.

8.5.1 Dipendenze di Dato (Data Dependencies - RAW)

C'è una reale dipendenza di dato se il flusso di informazioni lo richiede:

- l'istruzione i produce un risultato che viene letto/usato dall'istruzione j (Read After Write - RAW).
- c'è una dipendenza transitiva (es. j dipende da k, che a sua volta dipende da i).

8.5.2 Dipendenze di Nome (Name Dependencies)

Si verificano quando due istruzioni utilizzano lo stesso registro o locazione di memoria (stesso *nome*), ma **non esiste un reale flusso di dati** tra le due istruzioni. Creano delle "false dipendenze" che fermano inutilmente la pipeline. Si dividono in due categorie:

Antidipendenza (WAR - Write After Read)

L'istruzione j **scrive** su un registro o locazione di memoria che l'istruzione i **legge**, e l'istruzione i deve essere eseguita prima.

Esempio 8.4.
```
fadd.d f4, f0, f2   // i legge f0
...
fld    f0, -8(x1)   // j scrive su f0
```
Non c'è flusso di dati, ma se j finisse prima di i, modificherebbe il dato e i leggerebbe un valore errato.

Output Dependence (WAW - Write After Write)

Sia l'istruzione i che l'istruzione j **scrivono** nello stesso registro o locazione di memoria. L'ordine deve essere preservato per evitare che alla fine dell'elaborazione rimanga memorizzato un valore obsoleto.

8.5.3 Register Renaming (Rinominazione dei registri)

La soluzione ai problemi creati dalle dipendenze di nome è il **Register Renaming**. Poiché non c'è flusso di dati, rinominare il registro coinvolto nella seconda istruzione "slega" completamente i due blocchi di codice, rendendoli indipendenti e parallelizzabili. Nei processori moderni viene fatto dinamicamente in hardware; noi lo applichiamo staticamente (a mano/tramite compilatore) sfruttando i numerosi registri a disposizione (es. i 32 registri del RISC-V o i registri Floating Point).

Esempio 8.5. Consideriamo il seguente codice con una **Antidipendenza** (su `f6`):

```
fdiv.d f0, f2, f4
fadd.d f6, f0, f8   <-- Legge f6 per future operazioni (Output dep.) e lo scrive
fsd    f6, 0(x1)
fsub.d f8, f10, f14
fmul.d f6, f10, f8  <-- Antidipendenza su f8 (letto sopra) e Output dep. su f6
```

La `fdiv.d` è lenta. Senza accorgimenti, la `fmul.d` rischia di concludersi prima e sporcare `f6` prima che le operazioni in alto siano concluse (o viceversa). **Soluzione:** Sostituiamo `f6` con un registro temporaneo `S` nelle prime operazioni, eliminando la dipendenza di Output (WAW).

```
fdiv.d f0, f2, f4
fadd.d S, f0, f8   <-- Registro rinominato in S
fsd    S, 0(x1)    <-- Registro rinominato in S
fsub.d f8, f10, f14
fmul.d f6, f10, f8 <-- Ora questo blocco è indipendente
```

8.5.4 Dipendenze di Controllo

Avvengono quando un'istruzione dipende da un salto (branch).

Osservazione 8.3 (Spostamento delle istruzioni rispetto ai branch). Il professore evidenzia che quando si tenta di rischedulare spostando un'istruzione *prima* o *dopo* un branch per colmare uno stallo (ad esempio per riempire il *branch delay slot*), bisogna prestare assoluta attenzione a due cose:

1. **Data Flow:** Non modificare il flusso di dati. Se calcolo un valore prima di un salto condizionato, quel valore deve essere effettivamente utile e non andare a "sporcare" registri che serviranno in caso il salto non venga preso.

2. **Eccezioni:** Spostare un'istruzione, come un accesso alla memoria, prima di un branch potrebbe scatenare un page fault o un cache miss su un'istruzione che in realtà il programma non avrebbe mai dovuto eseguire se il salto avesse cambiato il flusso.

Capitolo 9

Memorie Cache

9.1 Introduzione e Motivazione

Le memorie cache sono memorie di piccole dimensioni ma ad altissima velocità, interposte tra il processore e la memoria principale. La necessità fondamentale di introdurre questa gerarchia deriva dall'enorme divario prestazionale tra il processore e la memoria. Attualmente, le prestazioni dei processori rispetto a quelle della memoria principale differiscono di un fattore di circa 1000. In termini pratici, nel tempo in cui il processore è in grado di eseguire migliaia di operazioni, la memoria principale è in grado di fornire una sola istruzione. Senza un sistema per mantenere dati e codice pronti e accessibili velocemente, il processore passerebbe la maggior parte del tempo in stallo, vanificando qualsiasi ottimizzazione architetturale, inclusa la pipeline.

Il costo energetico e temporale del calcolo odierno risiede prevalentemente nel trasferimento delle informazioni: elaborare un dato, per quanto l'operazione sia complessa, costa in termini di potenza circa mille volte meno rispetto al trasferire quel dato dalla memoria al processore.

9.1.1 Architettura di Von Neumann vs Harvard

Per comprendere l'integrazione della memoria nel sistema, è fondamentale richiamare la differenza tra le architetture di base:

- **Architettura Von Neumann**: Dati e istruzioni (codice, sub-routine, funzioni) risiedono in una singola memoria unificata. Il processore accede alla stessa memoria per prelevare sia il codice da eseguire sia i dati da elaborare.

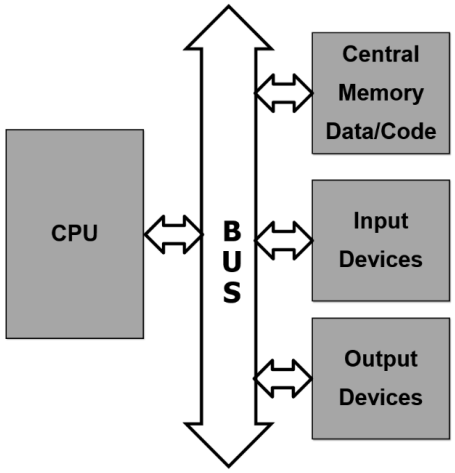


Figura 9.1: Architettura di Von Neumann: un unico spazio di memoria per dati e istruzioni.

- **Architettura Harvard**: Prevede memorie fisicamente separate per i dati e per il codice. Il processore può accedere simultaneamente alle due memorie attraverso bus distinti.

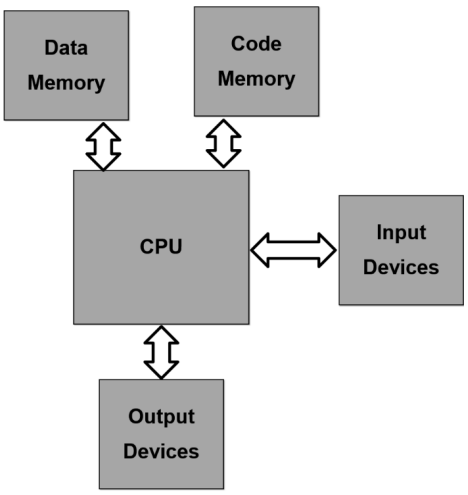


Figura 9.2: Architettura di Harvard: memorie separate per dati e istruzioni.

Nei sistemi moderni e specialmente nei processori RISC (come RISC-V e ARM) che implementano una pipeline, l'architettura Harvard è essenziale al livello della cache (Cache L1 Dati e Cache L1 Istruzioni separate). Ciò permette di eseguire contemporaneamente il fetch di un'istruzione e la lettura/scrittura di un dato, evitando conflitti strutturali che costringerebbero la pipeline a stallare.

9.2 Gerarchia delle Memorie e Principio di Località

Tra la memoria di immagazzinamento di massa (Hard Disk/SSD) e il processore, viene interposta una gerarchia di memorie per garantire un accesso rapido alle informazioni.

Analizzando il sistema partendo dall'interno verso l'esterno, troviamo:

1. **Registri della CPU**: Elementi di memoria interni (flip-flop) a cui la CPU accede alla massima frequenza di sistema (centinaia di picosecondi). La loro capacità è molto

limitata, tipicamente intorno a un kilobyte (es. 32 registri da 32 bit più i registri di controllo).

2. **Cache L1 (Livello 1)**: Memoria molto veloce (accesso in 1-2 nanosecondi) e di dimensioni ridotte (es. 64 KB). Tipicamente divisa in Dati e Istruzioni.

3. **Cache L2 (Livello 2) e L3 (Livello 3)**: Memorie progressivamente più grandi (da centinaia di KB a diversi MB) ma via via più lente.

4. **Memoria Principale (RAM)**: Memoria di lavoro capiente (diversi GB), ma con tempi di accesso lenti, nell'ordine dei 50-100 nanosecondi.

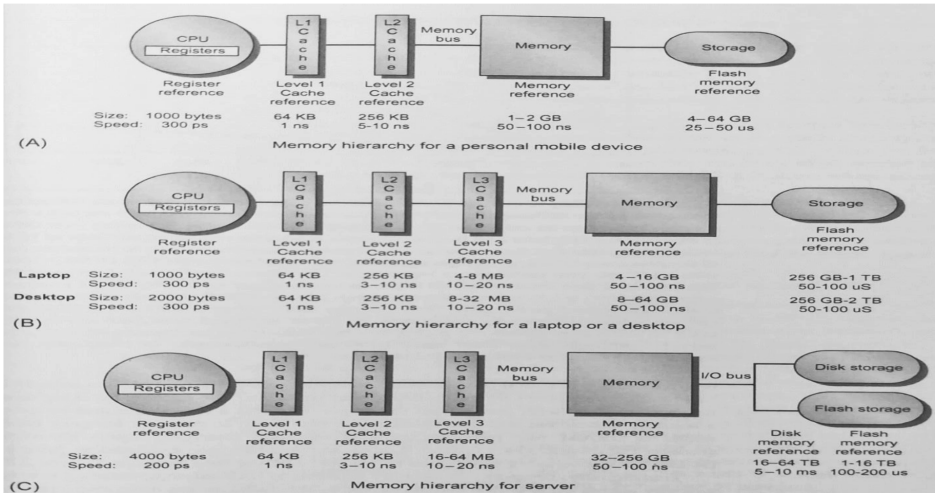


Figura 9.3: Gerarchia delle memorie: tempi di accesso e capacità per dispositivi mobili, laptop e server.

L'assunzione alla base dell'architettura gerarchica è il **principio di inclusività**: tutte le informazioni contenute nei registri dovrebbero trovarsi nella cache L1; tutte le informazioni della cache L1 devono essere contenute nella cache L2, e così via fino alla memoria principale. Se un dato non è in L1 (situazione di *miss*), il sistema lo cerca in L2, poi in L3, con un progressivo e drastico calo delle prestazioni a ogni salto.

Osservazione 9.1 (Inizializzazione nei Simulatori). Quando si analizzano le prestazioni tramite un simulatore, le prime fasi di avvio (es. la routine `_start`) comportano numerosi *cache miss* poiché la cache è inizialmente vuota. Per valutare il tempo reale di un algoritmo, è opportuno sottrarre i colpi di clock dell'inizializzazione (circa 208 cicli in alcuni simulatori) o assicurarsi di iniziare il conteggio dal `main`, operando una lettura preliminare dei dati per caricarli in cache.

9.2.1 Principio di Località

L'efficacia della cache si basa sulla località dei riferimenti osservata nell'esecuzione della maggior parte dei programmi. Si manifesta in due forme:

- **Località Temporale**: Se al tempo t un programma accede a una determinata cella di memoria, è altamente probabile che acceda nuovamente alla stessa cella in un istante prossimo $t_0 + \Delta t$. Per questo motivo, quando un dato viene letto, esso viene mantenuto in cache.

- **Località Spaziale**: Se al tempo t_0 il programma accede all'indirizzo x, è molto probabile che a breve accederà agli indirizzi contigui $x \pm e$ (ad esempio, gli elementi successivi di un vettore). Per sfruttare questa proprietà, quando si verifica un *miss*, la cache non carica dalla memoria soltanto il byte richiesto, ma l'intera **linea di cache** (es. 32 o 64 byte contigui).

9.2.2 Prestazioni e Hit Ratio

Per quantificare le prestazioni del sistema di memoria, si definiscono i seguenti parametri:

- h: **Hit ratio** (tasso di successo), ovvero la percentuale di volte in cui il dato richiesto viene trovato in cache.
- C: Tempo di accesso alla cache.
- M: Tempo di accesso alla memoria principale (in caso di *miss*).

Il tempo medio di accesso alla memoria (t_{ave}) si calcola come:

$$t_{\text{ave}} = h \cdot C + (1 - h) \cdot M$$

Valori normali di h si attestano sull'80%-90% ($0.8 - 0.9$). Se, per esempio, abbiamo un hit ratio dell'80% ($h = 0.8$), un tempo di accesso alla cache di 2 ns (2×10^{-9} s) e un tempo di accesso alla RAM ipotizzato, per enfatizzare il ritardo, a 50μs, il calcolo sarà:

$$T_{\text{avg}} = 0.8 \times (2 \cdot 10^{-9}) + 0.2 \times (50 \cdot 10^{-6}) \approx 10.001 \mu s$$

Si nota come anche una piccola frazione di miss (0.2) domini completamente il tempo medio di accesso a causa dell'enorme scarto temporale tra C e M.

9.3 Organizzazione Interna della Cache

Come quasi tutti i dispositivi digitali, la cache è divisa in una parte dedicata ai dati e una parte di controllo. Più specificamente, una memoria cache è strutturata in **linee** (o blocchi) ed è costituita da due componenti principali:

1. **Array dei Dati (Data Array)**: Contiene i blocchi di memoria effettivi caricati dalla memoria principale. Ogni linea contiene una quantità B di byte (es. 4, 32 o 128 byte).
2. **Array di Direttorio (Tag Array)**: Contiene le informazioni necessarie per identificare univocamente da quale indirizzo della memoria principale provengono i dati salvati nella linea corrispondente.

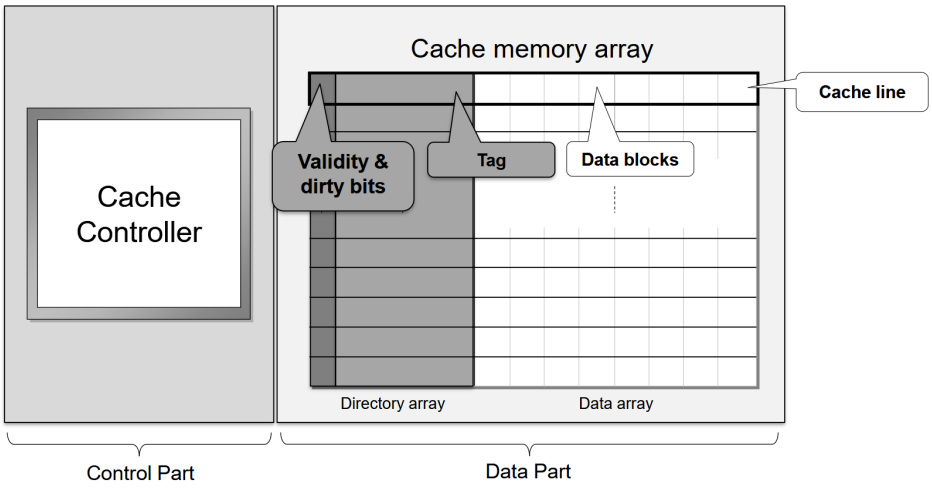


Figura 9.4: Organizzazione interna della cache: array dei dati e array di direttorio con tag e bit di validità.

Oltre ai dati e al tag, ogni linea di cache contiene alcuni bit di controllo fondamentali:

- **Bit di Validità (V - Validity bit)**: Indica se il dato presente nella linea è valido. All'accensione del sistema (power-on) o in caso di reset, la memoria cache contiene valori casuali (dati "sporchi"). Per evitare che la CPU legga informazioni scorrette, il bit V viene inizializzato a 0. Quando un blocco di memoria valido viene caricato nella linea, V viene settato a 1.
- **Dirty bit (D)**: Ricorda se il blocco caricato in cache è stato modificato rispetto alla copia presente nella memoria principale (utilizzato nelle politiche di *write-back*). Se D = 1, i byte in cache sono più recenti e dovranno essere riscritti in memoria principale prima che la linea venga sovrascritta.

9.3.1 Ricerca di un blocco dati nella Cache

Per comprendere se un dato è presente in cache, la logica di controllo riceve l'indirizzo a 32 bit generato dalla CPU (indirizzo di memoria) e lo suddivide in tre campi: **Tag**, **Indice (Index)** e **Offset**.



Figura 9.5: Scomposizione dell'indirizzo della CPU in Tag, Index e Offset per la ricerca nella cache.

1. **Offset**: Sono i bit meno significativi dell'indirizzo. Servono per individuare il byte specifico all'interno della linea di cache. Se una linea è ampia 2^n byte, l'offset sarà composto da n bit. Questo campo non viene salvato nel direttorio, poiché indica solo una posizione relativa all'interno del blocco dati già caricato.
2. **Indice (Index)**: Determina a quale linea (o insieme di linee, detto *set*) della cache accedere. Se la cache possiede 2^m insiemi, l'indice utilizzerà m bit.

3. **Tag**: Costituisce la parte più significativa dell'indirizzo ed è l'effettiva informazione che viene salvata nell'array di direttorio. La sua dimensione è data dai restanti bit: $32 - (n + m)$.

Il procedimento di ricerca avviene in questo modo:

1. La cache usa l'Indice per selezionare la riga (set) corrispondente.
2. Controlla il Bit di Validità di quella riga. Se $V = 0$, si ha immediatamente un *miss*.
3. Se $V = 1$, confronta il Tag memorizzato nel direttorio della riga con il campo Tag dell'indirizzo richiesto.
4. Se corrispondono, si ha un **Hit** (e l'offset estrae il byte richiesto); se non corrispondono, si ha un **Miss**.

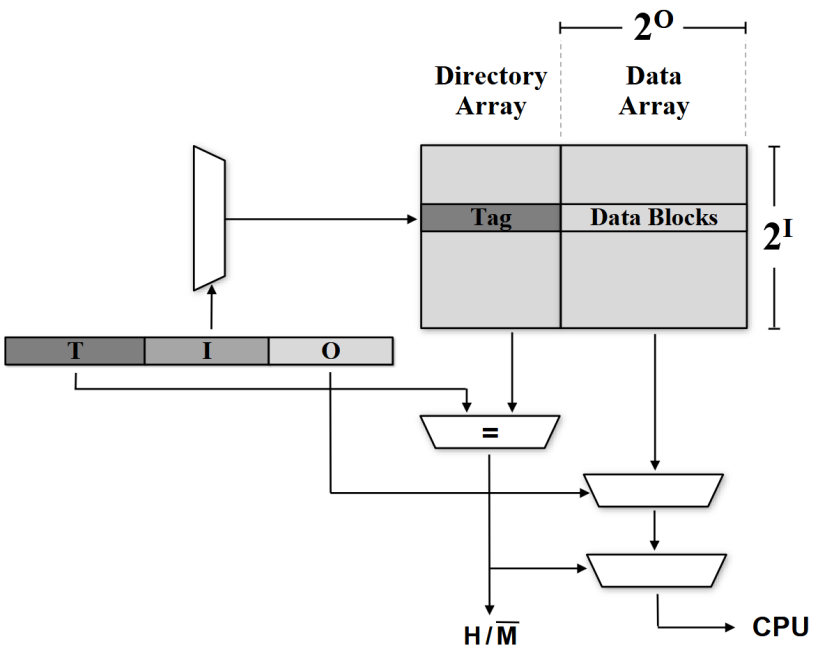


Figura 9.6: Procedura di ricerca di un dato nella cache: utilizzo dell'Indice, verifica del Bit di Validità e confronto del Tag.

Osservazione 9.2 (Dimensione Reale della Cache). Quando un costruttore vende una cache, ad esempio, da 64 KB, si riferisce unicamente alla capacità dell'Array dei Dati. La dimensione fisica reale implementata sul chip è molto più grande, poiché a ogni linea va sommato lo spazio per salvare il Tag, il bit V e il bit D, moltiplicato per il numero totale di linee.

9.3.2 Posizionamento della Cache nel Sistema

La cache viene collocata logicamente tra la CPU e il bus di sistema, piuttosto che tra il bus e la memoria principale. Questo posizionamento strategico comporta che, in caso di *hit*, il processore legga o scriva il dato senza alcun transito di informazioni sul bus di sistema. Ciò riduce drasticamente il carico sul bus, lasciandolo libero per periferiche e DMA, e risulta indispensabile nelle architetture multiprocessore per permettere ad ogni core di lavorare indipendentemente sui propri dati.

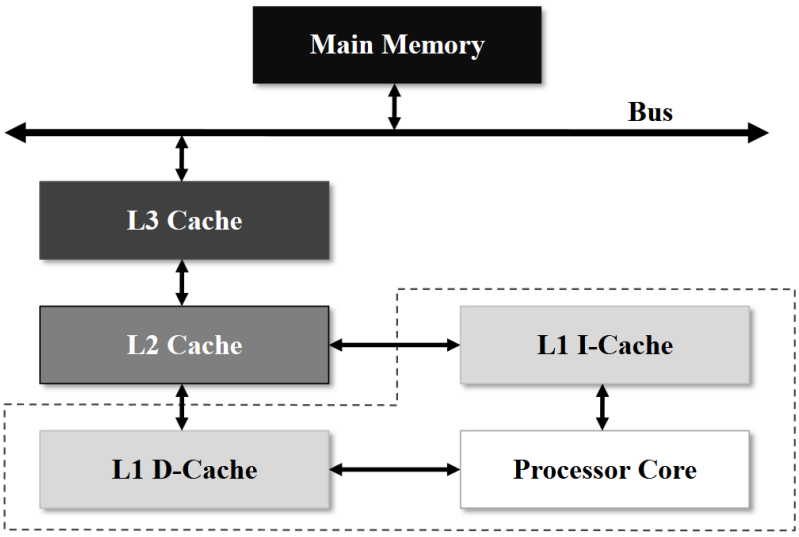


Figura 9.7: Posizionamento della cache tra CPU e bus di sistema, riducendo il traffico sul bus in caso di hit.

9.4 Tecniche di Mappatura (Mapping)

Il meccanismo di mapping definisce in quale linea della cache un determinato blocco di memoria principale deve essere posizionato. Questo processo deve essere rapido e richiedere un hardware a basso costo.

9.4.1 Mappatura Diretta (Direct Mapping)

Nella mappatura diretta, ogni blocco di memoria i viene associato staticamente e univocamente a una linea (o set) k della cache mediante l'operazione di modulo:

$$k = i \bmod N$$

dove N è il numero totale di linee della cache.

In termini hardware, questa operazione non richiede una reale divisione, poiché il calcolo corrisponde semplicemente al prelievo dei bit meno significativi (l'Indice) dell'indirizzo del blocco.

Vantaggi: Il meccanismo è facilissimo da implementare in hardware.
Svantaggi (Problema del Thrashing): Se il programma accede frequentemente a due indirizzi di memoria che, pur differendo nei bit più alti, presentano gli stessi bit di indice (es. l'indirizzo 4 e l'indirizzo 12 modulo 8 restituiscono entrambi la posizione 4), essi competeranno per la stessa unica riga della cache. Ad ogni lettura ci sarà un *miss* che obbligherà il sistema a sfrattare il dato precedente, abbattendo le prestazioni.

9.4.2 Mappatura Associativa agli Insiemi (Set Associative)

Per arginare il problema della competizione per le righe, la cache viene divisa in un numero S di **insiemi (Set)**, ognuno dei quali contiene un numero W di linee detto **vie (Ways)**. Si definisce quindi l'equazione:

$$S = \frac{N}{W}$$

dove N è il numero totale di linee. Un blocco di memoria i viene ora associato al set k:

$$k = i \bmod S$$

Una volta calcolato il set k, il blocco di memoria può essere salvato in *qualsiasi* delle W linee disponibili in quel set. Se abbiamo una cache a 2 vie ($W = 2$), l'indirizzo 4 e l'indirizzo 12 calcoleranno lo stesso indice, ma potranno convivere tranquillamente occupando le due linee del medesimo set.

Effetti dell'Associatività sull'Indirizzo: Aumentare l'associatività a parità di dimensione della cache (Array Dati fisso) comporta che il numero di Set (S) diminuisca. Di conseguenza:

- La dimensione del campo **Indice** diminuisce (perché ci sono meno set da indicizzare).
- La dimensione del campo **Tag** aumenta (per compensare i bit persi dall'indice).
- Poiché il campo Tag si allunga e va salvato per ciascuna delle N linee (le linee restano invariate in numero assoluto), **la dimensione fisica reale della memoria di controllo aumenta**.

9.4.3 Mappatura Completamente Associativa (Fully Associative)

In questo caso, l'Indice viene eliminato del tutto. Un blocco di memoria può risiedere in *qualsiasi* linea della cache. Questo schema offre la massima flessibilità e minimizza gli scarti forzati (no conflitti di set), ma richiede un hardware estremamente complesso, poiché al momento della ricerca la CPU deve confrontare simultaneamente il Tag con *tutte* le linee della cache (memoria CAM - Content Addressable Memory). Il campo Tag in questo caso copre quasi tutto l'indirizzo, escluso l'Offset.

9.5 Algoritmi di Rimpiazzamento

Quando si ha un *miss* e tutte le vie del Set calcolato sono già occupate da dati validi, è necessario scegliere quale blocco sfrattare per fare spazio al nuovo dato. Le strategie principali sono:

- **LRU (Least Recently Used)**: Rimpiazza la linea usata meno di recente. Richiede di memorizzare la "storia" degli accessi (es. tramite contatori), risultando costoso in termini di bit addizionali e logica.
- **FIFO (First-In First-Out)**: Rimuove semplicemente il dato entrato per primo (il più vecchio), a prescindere da quanto frequentemente venga usato.
- **LFU (Least Frequently Used)**: Teoreticamente il più efficace, guarda alla frequenza di utilizzo assoluta, ma molto complesso da implementare.
- **Random**: Semplice ed efficace, sostituzione casuale.

9.5.1 Approccio FIFO con Seconda Chance (Clock Bit)

Per mitigare il problema del FIFO puro, che potrebbe espellere un dato molto vecchio ma costantemente utilizzato, si adotta spesso una politica con "Seconda Opportunità". Si associa a ogni linea un **Clock bit** che viene settato a 1 qualora vi sia un "hit" su quella riga (ovvero il dato viene riutilizzato). Quando il sistema deve espellere una riga, un puntatore circolare esamina le vie in ordine d'ingresso:

- Se il Clock bit è 1, il sistema lo porta a 0, non espelle il dato (dandogli una seconda chance), e fa avanzare il puntatore.
- Se il Clock bit è 0, il dato viene sfrattato.

9.5.2 Algoritmo pseudo-LRU (pLRU)

Poiché un vero algoritmo LRU richiederebbe, per un'architettura a 4 vie, 8 bit di storia per ogni Set (diventando troppo costoso in hardware), si adotta una sua approssimazione efficiente basata su un albero binario decisionale, aggiungendo soli 3 bit di storia (a_0, b_0, b_1) per set (nel caso di 4 vie).

I bit indicano il percorso nell'albero per trovare la "foglia" (via) usata meno di recente. Quando una via viene scritta o letta, i bit sul percorso che conducono a essa vengono invertiti, indicando di "non andare più da quella parte" per il prossimo rimpiazzo. Ad esempio, se i 3 bit partono da stato ‘0 0 0‘, e vengono generati miss consecutivi (A, B, C, D):

1. Arriva il dato **A**: l'albero suggerisce la via 0. Il dato viene allocato e i nodi attraversati invertono stato.
2. Arriva il dato **B**: i nodi ora puntano alla via 1.
3. Arriva **C**: le inversioni precedenti fanno deviare la radice verso l'altro ramo, puntando alla via 2.
4. Arriva **D**: viene indicata la via 3.

È molto più compatto, anche se in certi pattern anomali di lettura potrebbe finire col preservare elementi non necessari.

9.6 Gestione delle Scritture e Coerenza

9.6.1 Aggiornamento della Memoria Principale

Quando il processore scrive un dato in cache, sorge il problema di quando aggiornare la memoria principale per allinearla al nuovo valore. Le due soluzioni sono:

- **Write-Back**: Il dato viene aggiornato in memoria principale *esclusivamente* nel momento in cui la linea corrispondente deve essere sfrattata dalla cache. Per ricordare che il dato è stato alterato rispetto alla RAM, la linea imposta a 1 il suo **Dirty bit**.
  Pro: Riduce enormemente gli accessi e l'occupazione del bus.
  Contro: Sfrattare un dato costa tempo aggiuntivo (deve essere fisicamente riscritto in RAM). In sistemi multiprocessore, genera problemi di coerenza.
- **Write-Through**: Ogni singola operazione di scrittura viene propagata *immediatamente* sia sulla cache che sulla memoria principale.
  Pro: La memoria è sempre aggiornata e coerente.
  Contro: Abbassa le prestazioni impegnando il bus, anche se limitato dal fatto che le letture statisticamente superano di molto le scritture.

Inoltre, se la scrittura del processore avviene su un indirizzo non presente in cache (**write miss**), si può decidere se:

- **Write-allocate**: Caricare l'intero blocco dalla memoria alla cache, modificarlo in cache e mantenerlo lì. Usato tipicamente col *Write-back*.
- **No write-allocate**: Inviare il dato direttamente in memoria senza caricarlo in cache. Usato solitamente col *Write-through*.

9.6.2 Coerenza della Memoria (Multiprocessore) e Protocollo MESI

In architetture HPC (High Performance Computing) e nei classici sistemi cluster o multicore che condividono la memoria centrale (e una o più cache di basso livello, es. L3 condivisa tra processori aventi L1 e L2 private), il *Write-back* fa sì che il core A abbia un dato aggiornato nella sua cache L1 privata che il core B non vede, causando letture corrotte (incoerenza).

Per mantenere la coerenza, ogni linea di cache, in aggiunta ai bit V e D, traccia il suo stato di condivisione. Il protocollo più celebre è il **MESI**:

- **M - Modified**: La linea è stata modificata dal core locale ed è posseduta in modo esclusivo (il dato in RAM non è aggiornato).
- **E - Exclusive**: La linea è coerente con la RAM ed è presente esclusivamente in questa singola cache locale.
- **S - Shared**: La linea è stata letta ma non modificata; anche altre cache la possiedono e i valori combaciano tutti con la RAM.
- **I - Invalid**: Se il core A scrive su un blocco che in precedenza era condiviso (Shared), invia sul bus un segnale che forza il blocco corrispondente in tutte le altre cache a marcare la linea come Invalida. Se il core B chiederà il dato, dovrà prima costringere il core A ad aggiornare la memoria o passargli direttamente il nuovo valore.

9.7 Esempi Calcolati e Strutturazione Fisica

In questa sezione si illustrano esplicitamente i calcoli atti a dimensionare tag, indice e offset, necessari per risalire alla reale grandezza fisica di una memoria cache in aggiunta all'array dei dati.

9.7.1 Esempio 1: Cache Direct Mapped

Assumiamo le seguenti caratteristiche:

- Capacità Array Dati: 64 kB
- Mappatura: Direct Mapping
- Dimensione linea/blocco: 4 Byte
- Indirizzi su 32 bit

Procedimento:

1. **Calcolo dell'Offset**: Poiché ogni linea contiene 4 Byte (2^2), servono **2 bit** di Offset per indirizzare i singoli byte all'interno del blocco.
2. **Calcolo dell'Indice**: Il numero di linee totali è la dimensione totale diviso la capacità della singola linea:

$$\frac{64 \text{ kB}}{4 \text{ Byte}} = \frac{2^6 \cdot 2^{10}}{2^2} = 2^{14} \text{ linee (16k linee)}$$

   L'Indice richiederà quindi **14 bit**.

3. **Calcolo del Tag**: Si sottraggono i bit di Indice e Offset dall'indirizzo:

$$32 - 14 - 2 = \textbf{16 bit} \text{ per il Tag}$$

La dimensione reale fisica di questa cache non è 64 kB, bensì bisogna sommare ad ogni blocco (linea) di 4 Byte (32 bit) anche la dimensione del Tag (16 bit), tralasciando per semplicità i bit V e D:

$$\text{Capacità Totale} = 2^{14} \text{ linee} \times (32 \text{ bit dati} + 16 \text{ bit tag}) = 2^{14} \times 48 \text{ bit} = 768 \text{ kbit}$$

Dividendo per 8, otteniamo **96 kByte**, un valore sensibilmente superiore al dato nominale (64 kB).

9.7.2 Esempio 2: Cache Set-Associative in un Layout di Memoria Misto

Consideriamo il seguente scenario, tipico di sistemi embedded, in cui il processore richiede il calcolo dei parametri di una cache con queste specifiche:

- Capacità Array Dati: 4 KB
- Mappatura: 2-vie Set Associative (Associatività = 2)
- Lettura da memoria: 32 Byte per operazione. La linea di cache conterrà quindi blocchi di 32 Byte.
- Indirizzi su 32 bit

Procedimento:

1. **Offset**: Per contenere 32 Byte, l'Offset necessita di **5 bit** ($2^5 = 32$).
2. **Linee totali e Indice**: Calcoliamo il numero totale di linee:

   $$\frac{4 \text{ KB}}{32 \text{ Byte}} = \frac{4096}{32} = 128 \text{ linee totali}$$

   Siccome la cache è a 2 vie, le linee vengono raggruppate in insiemi (Set) da due.

   $$\text{Numero di Set} = \frac{128 \text{ linee}}{2} = 64 \text{ Set}$$

   Per indirizzare 64 Set (2^6) servono **6 bit** di Indice.
3. **Tag**: Sottraiamo Indice e Offset:

   $$32 - 5 - 6 = \textbf{21 bit} \text{ per il Tag}$$

La memoria extra aggiunta per il salvataggio dei Tag sarà pari a:

$$128 \text{ linee (totali)} \times 21 \text{ bit} = 2688 \text{ bit} = 336 \text{ Byte}$$

Sommando la logica supplementare a una cache già molto piccola come in questo esempio, l'incremento di percentuale dello spazio richiesto è piuttosto gestibile, sebbene sia richiesto un bit V per ognuna delle 128 linee.

9.7.3 Esempio 3: Cache Associativa a 4-vie

Parametri:

- Dati: 4 KB
- Mappatura: 4-vie Set Associative
- Linea: 32 Byte
- Indirizzo: 32 bit

Procedimento:

1. **Offset**: $2^5 = 32 \rightarrow$ **5 bit**.
2. **Linee totali**: 4KB/32 = 128 linee totali.
3. **Indice**: 128 linee / 4 vie = 32 Set (2^5). Indice = **5 bit**.
4. **Tag**: $32 - (5 + 5) =$ **22 bit**.

Calcolo dell'area logica di controllo (Direttorio):

- Tag Bits totali: $128 \times 22 = 2816$ bit.
- Bit di Validità (V): 128 bit.
- Dirty bits (D): 128 bit.
- Bits per il pLRU Replacement: l'algoritmo pseudo-LRU per 4 vie richiede 3 bit per Set. Pertanto i 32 Set avrebbero $32 \times 3 = 96$ bit.
  Nota: nelle slide, un'approssimazione indica prudenzialmente 2 bit per linea, ottenendo $128 \times 2 = 256$ *bit. Adotteremo per coerenza il risultato didattico fornito dal professore nelle slide per il calcolo finale.*

Dimensione totale memoria cache:

$$4 \text{ KB} + \text{Bit di Controllo}$$

$$32768 \text{ bit (Dati)} + 2816(\text{Tag}) + 128(\text{V}) + 128(\text{D}) + 256(\text{pLRU}) = 35584 \text{ bit} = 4448 \text{ Byte}$$

Immagini da recuperare

- **Slide 4 - Diagramma dei livelli di memoria**: Grafico a blocchi rappresentante gerarchia per Mobile, Laptop e Server. Da inserire nella Sezione "Gerarchia delle Memorie".
- **Slide 9 e 10 - Organizzazione Interna**: Schema dei blocchi Data Array e Tag/Directory Array, incluse V e D flag. Da inserire nella Sezione "Organizzazione Interna della Cache".
- **Slide 15 - Scomposizione CPU Address**: Rettangolo raffigurante i 32 bit suddivisi in Tag, Index e Offset. Da posizionare in "Ricerca di un blocco dati nella Cache".

- **Slide 28 e 29 - Set Associative Mapping**: Mappatura dei blocchi di memoria su cache suddivisa a insiemi e vie, mostrato con frecce illustrative. Da inserire sotto "Mappatura Associativa agli Insiemi".
- **Slide 36 a 45 - Pseudo-LRU (pLRU)**: Evoluzione temporale (sequenza A-E) di un albero binario a 3 nodi decisionali. Da inserire sotto "Algoritmo pseudo-LRU".
- **Slide 61 a 73 - Schema layout di Memoria Misto**: Sequenza animata/strutturata che illustra il calcolo dei bit per tag/index e layout della memoria segmentata associato al codice assembly. Da inserire in "Esempio 2".

Problemi da verificare

- **Calcolo dei bit di storia per pLRU (Esempio 3)**: Nelle slide (Slide 74) è riportato il dato “LRU Replacement = 2-bits per line = 256 bits”. L’algoritmo pLRU ad albero binario spiegato nella Sezione 5.2, implementato per una cache a 4 vie, necessiterebbe di 3 bit totali per ciascuno dei 32 set (quindi 96 bit e non 256). Si consiglia di verificare col docente se la dicitura “2-bits per line” fa riferimento ad un algoritmo di history replacement diverso dall’albero binario, o se si tratti di un arrotondamento cautelativo generico per il design.

Capitolo 10

Branch prediction

Questo capitolo è riservato allo studio della predizione dei salti.

Capitolo 11

Dynamic scheduling

Questo capitolo è riservato allo studio dello scheduling dinamico.

Capitolo 12

Hardware speculation

Questo capitolo è riservato allo studio della speculazione hardware.

Capitolo 13

Sistemi di ingresso e uscita

13.1 Periferiche

13.2 Interrupt

13.3 Accesso diretto alla memoria

Capitolo 14

Esercizi

14.1 Esercizi svolti

Esempio 14.1. *Testo dell'esercizio e relativa soluzione.*

14.2 Esercizi da svolgere

1. Inserire qui il testo del primo esercizio.
2. Inserire qui il testo del secondo esercizio.

Capitolo 15

VLIW

Questo capitolo è riservato allo studio delle architetture VLIW.

Capitolo 16

Riepilogo

Riassumere in questa sezione i concetti principali del corso.

Capitolo 17

ESERCIZI

Capitolo 18

Esercitazione: Analisi delle Prestazioni e Pipeline

In questo capitolo analizziamo un esercizio completo, tipico di una prova d'esame, che richiede di valutare le prestazioni di un frammento di codice assembly in esecuzione su una specifica architettura di processore. L'obiettivo è calcolare il numero esatto di colpi di clock necessari per l'esecuzione del programma, analizzando il comportamento della pipeline, le dipendenze tra i dati (hazard) e le relative penalità (stalli).

18.1 Definizione dell'Architettura del Processore

Il sistema di riferimento per questo esercizio è un processore RISC-V con architettura RV32G, dotato di supporto per le operazioni in virgola mobile (Floating-Point, FP) a singola precisione. Le caratteristiche specifiche delle unità funzionali e della pipeline sono le seguenti:

- **ALU Intera (Integer ALU):** Richiede 1 colpo di clock per completare l'operazione nella fase di esecuzione (Execute).
- **Memoria Dati (Data memory):** L'accesso richiede 1 colpo di clock. Si assume, per semplicità, che tutti i dati siano presenti in cache (nessun miss).
- **Unità Moltiplicatore FP:** È un'unità dotata di pipeline (pipelined) strutturata in 8 stadi.
- **Unità Aritmetica FP (Somma/Sottrazione):** È dotata di pipeline strutturata in 4 stadi.
- **Unità Divisore FP: Non** è dotata di pipeline e richiede 8 colpi di clock per completare un'operazione. Questo significa che, finché l'operazione di divisione non è conclusa, l'unità non può accettare nuove istruzioni.
- **Branch Delay Slot:** Presente (1 colpo di clock), ma **disabilitato**. Di conseguenza, i salti genereranno una penalità se presi.
- **Forwarding:** Abilitato. I dati calcolati possono essere passati direttamente alle istruzioni successive senza attendere la scrittura nel banco registri (fase di Write Back).
- **Esecuzione Fuori Ordine (Out-of-Order):** È possibile completare la fase di Execute (EXE) delle istruzioni in modalità "fuori ordine". Questo permette ad alcune istruzioni di avanzare e accedere alla memoria o calcolare indirizzi anche se le istruzioni precedenti sono in stallo.

18.2 Analisi del Codice Assembly

Il programma da analizzare esegue un ciclo iterativo (loop) che elabora cento elementi per volta, leggendo da tre vettori in memoria (`V1`, `V2`, `V3`), eseguendo operazioni aritmetiche su di essi e salvando i risultati in altri tre vettori (`V4`, `V5`, `V6`). Le operazioni logiche del ciclo sono:

1. Somma: `v4[i] = v1[i] + v2[i]`
2. Moltiplicazione: `v5[i] = v2[i] * v3[i]`
3. Divisione: `v6[i] = v1[i] / v3[i]`

Di seguito è riportato il codice assembly corrispondente:

```
.text
main:
    addi  x7, x0, 100       # Inizializza il contatore del ciclo a 100
    la    x1, V1            # Carica l'indirizzo base del vettore V1
    la    x2, V2            # Carica l'indirizzo base del vettore V2
    la    x3, V3            # Carica l'indirizzo base del vettore V3
    la    x4, V4            # Carica l'indirizzo base del vettore V4
    la    x5, V5            # Carica l'indirizzo base del vettore V5
    la    x6, V6            # Carica l'indirizzo base del vettore V6

loop:
    flw   f1, 0(x1)         # Carica v1[i]
    flw   f2, 0(x2)         # Carica v2[i]
    fadd.s f4, f1, f2       # f4 = v1[i] + v2[i]
    flw   f3, 0(x3)         # Carica v3[i]
    fsw   f4, 0(x4)         # Salva il risultato della somma in V4
    fmul.s f5, f2, f3       # f5 = v2[i] * v3[i]
    fsw   f5, 0(x5)         # Salva il risultato della moltiplicazione in V5
    fdiv.s f6, f1, f3       # f6 = v1[i] / v3[i]
    fsw   f6, 0(x6)         # Salva il risultato della divisione in V6

    # Aggiornamento degli indici (4 byte per numero FP a singola precisione)
    addi  x1, x1, 4
    addi  x2, x2, 4
    addi  x3, x3, 4
    addi  x4, x4, 4
    addi  x5, x5, 4
    addi  x6, x6, 4

    addi  x7, x7, -1        # Decrementa il contatore del ciclo
    bnez  x7, loop          # Salta a 'loop' se x7 è diverso da zero
    nop                     # Istruzione vuota
```

18.3 Calcolo dei Colpi di Clock

Il nostro obiettivo è tracciare il percorso di queste istruzioni all'interno della pipeline (composta tipicamente dalle fasi: Fetch, Decode, Execute, Memory, Write Back) per contare il totale dei colpi di clock richiesti.

Figura 18.1: Griglia temporale (diagramma di Gantt della pipeline) per il tracciamento dell'esecuzione delle singole istruzioni (da recuperare dalle slide).

18.3.1 Fase di Inizializzazione (Fuori dal ciclo)

Prima del ciclo, il programma esegue le istruzioni di inizializzazione. Dobbiamo calcolare il loro peso in termini di cicli di clock.

- **Prima istruzione (`addi x7, x0, 100`):** Essendo la prima istruzione dopo il reset, deve attraversare tutte le 5 fasi della pipeline senza sovrapposizioni. Il suo contributo diretto è di **5 colpi di clock**. Va ricordato che queste somme sui registri interi utilizzano la ALU intera e richiedono un solo ciclo nella fase di Execute.
- **Istruzioni `la` (Load Address):** Il professore fa notare che le direttive `la` sono *pseudo-istruzioni*. Il compilatore le traduce in due istruzioni macchina reali. Poiché non generano dipendenze complesse tra loro e il forwarding è abilitato, possiamo considerare che ogni pseudo-istruzione `la` aggiunga **2 colpi di clock** alla latenza complessiva della pipeline.
- Avendo 6 istruzioni `la`, il loro costo totale è $6 \times 2 = 12$ cicli.

Il totale della fase di inizializzazione è pertanto:

$$\text{Cicli Inizializzazione} = 5 + 12 = 17 \text{ colpi di clock}$$

18.3.2 Tracciamento di una Iterazione del Ciclo (`loop`)

La parte fondamentale dell'esercizio consiste nel valutare gli stalli all'interno del corpo del ciclo. Analizziamo le istruzioni critiche e le loro interazioni.

Caricamento dati e Somma FP

Le prime due istruzioni sono load (`flw f1...` e `flw f2...`). Esse fluiscono regolarmente nella pipeline. La terza istruzione è `fadd.s f4, f1, f2`. Questa operazione presenta una dipendenza di dato RAW (Read After Write) con l'istruzione precedente: richiede `f2`, che è stato appena letto dalla memoria.

- Poiché il dato da memoria è disponibile solo alla fine dello stadio *Memory* della load, la `fadd.s` non può avviare la fase di *Execute* e deve **stallare per 1 colpo di clock**.
- Dopo lo stallo, l'unità aritmetica FP impiega 4 stadi di Execute.

Completamento Fuori Ordine ed Eccezioni Imprecise

La quarta istruzione è `flw f3, 0(x3)`. Il professore evidenzia un punto architetturale cruciale: questa istruzione non ha dipendenze. Essendo consentito il completamento "fuori ordine" per la fase EXE, questa istruzione può calcolare il suo indirizzo di memoria superando idealmente il blocco della somma precedente. Tuttavia, bisogna fare attenzione alla scrittura nel banco registri (Write Back). Se permettessimo la scrittura in memoria/registri fuori ordine, andremmo incontro al problema delle **eccezioni imprecise**: se l'istruzione precedente (`fadd.s`) dovesse generare un'eccezione, il processore si fermerebbe, ma lo stato architetturale risulterebbe alterato (perché `f3` è già stato sovrascritto dalla load). In questo specifico caso, trattandosi di una Load, il professore sceglie di far avanzare l'istruzione in modo che non contribuisca a ritardi aggiuntivi (costo 0 colpi di clock supplementari rispetto al flusso), rendendo il dato disponibile per le istruzioni future.

Store del risultato FP e Stalli Strutturali

L'istruzione successiva è `fsw f4, 0(x4)`. Qui si verificano due problemi distinti:

1. **Stallo per dipendenza (RAW):** L'istruzione deve attendere che `f4` sia calcolato dalla `fadd.s` precedente. Pertanto si ferma nella pipeline in attesa del dato.
2. **Stallo Strutturale:** Anche quando il dato è pronto, l'istruzione non può accedere allo stadio di *Memory* contemporaneamente all'istruzione precedente se quest'ultima la sta occupando (priorità all'istruzione più vecchia).

Il professore osserva che la fase di Execute per la store (che serve unicamente a calcolare l'indirizzo base + offset) può essere svolta prima, calcolando l'indirizzo in anticipo, permettendo così di sbloccare parzialmente la fase di *Decode* per le istruzioni successive e migliorando le prestazioni complessive.

Moltiplicazione e Divisione

L'istruzione `fmul.s f5, f2, f3` utilizza registri (`f2` e `f3`) ormai pronti (il dato `f3` subisce *forwarding* appena prelevato dalla cache). Entra regolarmente nella sua unità dedicata, che è in pipeline e impiega 8 stadi per l'esecuzione. La store successiva (`fsw f5`) dovrà inevitabilmente stallare in attesa della fine degli 8 stadi di esecuzione della moltiplicazione.

Arriviamo alla divisione: `fdiv.s f6, f1, f3`. Durante la lezione, il professore corregge un errore logico: pur avendo operandi indipendenti e già pronti, la divisione **deve stallare in fase di decodifica** perché la store precedente sta occupando la pipeline. Una volta superato questo blocco, entra nell'unità di divisione, che richiederà **8 colpi di clock**. Essendo l'unità di divisione non in pipeline, bloccherà la fase di Execute per quel tipo di operazione. Anche qui, la successiva `fsw f6` genererà numerosi stalli in attesa del risultato (`f6`).

Istruzioni intere e Salto (`Branch`)

Le 6 istruzioni `addi` intere per l'aggiornamento degli indici si susseguono regolarmente contribuendo a riempire i "buchi" lasciati dai lunghi stalli delle operazioni FP precedenti. L'aggiornamento del contatore è l'istruzione `addi x7, x7, -1`. A seguire troviamo il salto condizionato: `bnez x7, loop`.

Qui si presenta un hazard fondamentale: il salto ha bisogno di valutare il valore di `x7`, che viene calcolato dall'istruzione immediatamente precedente. Il processore valuta i salti nello stadio di *Decode*. Di conseguenza, il salto **deve stallare per 1 colpo di clock** per permettere all'istruzione `addi` di terminare l'Execute e fornire il dato tramite forwarding.

Inoltre, poiché il *branch delay slot* è **disabilitato**, quando il salto viene effettivamente "preso" (taken), l'istruzione successiva che è già stata fetchata (la `nop` o l'istruzione sottostante) deve essere "scartata" (flushata), causando la **perdita di 1 ulteriore colpo di clock**.

Il professore sceglie didatticamente di attribuire la "colpa" di questo colpo di clock perso direttamente all'istruzione di salto, che contribuirà quindi con **2 colpi di clock totali** al tempo di esecuzione di ogni singola iterazione.

18.3.3 Calcolo del Tempo Totale di Esecuzione

Svolgendo la tracciatura completa nella griglia temporale (Gantt), il professore determina che **una singola iterazione completa del ciclo costa esattamente 28 colpi di clock**.

Per calcolare il totale del programma (costituito da 100 iterazioni), la formula è:

$$\text{Cicli Ciclo Totali} = 100 \times 28 = 2800 \text{ colpi di clock}$$

Tuttavia, occorre fare una precisazione vitale per l'ultima iterazione. Durante l'iterazione numero 100, la condizione del salto fallisce (il salto *non* viene preso perché `x7` diviene 0). In questa specifica iterazione non si verifica il "flush" dell'istruzione successiva, pertanto si **risparmia un colpo di clock**. Il calcolo corretto per le iterazioni diventa quindi:

$$\text{Cicli Ciclo Effettivi} = (99 \times 28) + (1 \times 27) = 2800 - 1 = 2799$$

In alternativa, si possono calcolare i cicli per 100 iterazioni (2800) e sottrarre alla fine 1 ciclo.

Sommando la fase di inizializzazione iniziale (17 cicli), otteniamo il risultato finale:

$$\text{Totale Cicli} = \text{Fase Inizializzazione} + \text{Iterazioni del loop} - \text{Risparmio ultimo salto}$$

$$\text{Totale Cicli} = 17 + 2800 - 1 = \mathbf{2816} \text{ colpi di clock}$$

Nota: Durante la spiegazione orale, il professore nomina la cifra 2817 sommando 17 e 2800, specificando immediatamente dopo che "verrebbe un colpo di clock in meno" per via dell'ultimo salto non preso, confermando il numero 2816 finale.

18.4 Analisi degli Hazard (Domande Teoriche)

A completamento dell'esercizio, vengono proposte due domande teoriche sull'architettura. Entrambe si basano sull'individuazione di specifici hazard presenti nel codice.

18.4.1 Hazard sul registro `x7`

Domanda: Considerando le istruzioni `addi x7, x7, -1` e `bnez x7, loop`, che tipo di hazard è causato dall'uso del registro `x7` e come viene risolto?

Risposta: Si tratta di un **Hazard sui Dati di tipo RAW (Read After Write)**. L'istruzione di salto (`bnez`) deve leggere il contenuto del registro `x7` per prendere la decisione sul branch, ma tale valore viene scritto (modificato) dall'istruzione immediatamente precedente (`addi`). **Risoluzione:** L'hazard viene risolto inserendo uno **stallo (stall/bubble)** nella pipeline. Nello specifico, poiché il salto valuta la condizione in fase di Decodifica, deve arrestarsi (stallare) finché l'istruzione `addi` non conclude la fase di Execute e inoltra (forwarding) il nuovo valore aggiornato.

18.4.2 Hazard sul registro `F6`

Domanda: Considerando le istruzioni `fdiv.s f6, f1, f3` e `fsw f6, 0(x6)`, che tipo di hazard è causato dall'uso del registro `f6` e come viene risolto?

Risposta: Si tratta anch'esso di un **Hazard sui Dati di tipo RAW (Read After Write)**. L'istruzione di store (`fsw`) deve salvare in memoria il dato contenuto in `f6`, ma questo dato è il risultato dell'istruzione di divisione FP precedente, che richiede ben 8 colpi di clock per completarsi. **Risoluzione:** L'hazard si risolve tramite lo **stallo del processore**. L'istruzione di store deve rimanere bloccata all'interno della pipeline (dopo aver eventualmente calcolato l'indirizzo in fase di Execute) e attendere passivamente per diversi colpi di clock la conclusione degli 8 stadi dell'unità di divisione FP. Solo una volta ottenuto il dato tramite forwarding, la store potrà accedere alla fase di Memory per eseguire il salvataggio fisico del dato.